

Vector Databases & Embeddings

In This Edition

1	Overview --- What It Is	4
2	Why It Exists	5
	2.1 Why k-d trees fail in high dimensions	5
3	Why FAANG Cares	6
4	Core Concepts	6
	4.1 Semantic search vs. lexical search	6
	4.2 The vector space	7
	4.3 Exact kNN cost recap	7
5	Embeddings	7
	5.1 What a dense embedding is	8
	5.2 How models learn embeddings --- the training objective	8
	5.3 Dimensionality --- 384 vs 768 vs 1536 vs 3072	9
	5.4 Normalization --- unit vectors	9
	5.5 Choosing / fine-tuning a domain model	9
	5.6 Multimodal --- CLIP and shared spaces	10
	5.7 Embedding drift & the cost of re-embedding	10
6	Similarity & Distance Metrics	11
	6.1 Cosine similarity	11
	6.2 Dot product (inner product)	11
	6.3 The normalized → cosine = dot equivalence (know this cold)	11
	6.4 Euclidean / L2 distance	11
	6.5 Manhattan (L1)	12
	6.6 Which metric, when	12
7	Approximate Nearest Neighbor (ANN) Indexes	12
	7.1 HNSW --- Hierarchical Navigable Small World (the default)	12
	7.2 IVF --- Inverted File (cluster-and-probe)	13
	7.3 Product Quantization (PQ) --- compressing vectors	14
	7.4 LSH --- Locality-Sensitive Hashing	15
	7.5 Tree-based --- Annoy	15
	7.6 ScaNN --- Anisotropic quantization (Google)	16
	7.7 ANN comparison table	16
8	The Recall / Latency / Memory Trade-off	17
	8.1 Recall@k --- the metric	17
	8.2 How parameters tune the triangle	17
9	Building a RAG / Semantic-Search Pipeline	18
	9.1 Chunking strategies	18
	9.2 Embed + upsert	19
	9.3 Retrieval top-k	19
	9.4 Metadata filtering --- pre- vs post-filter (a real correctness/perf trap)	19
	9.5 Hybrid search --- dense + sparse (BM25) with RRF	20
	9.6 Reranking --- the two-stage retrieve → rerank pattern	20
	9.7 Putting it together --- query pipeline	21
10	Vector Database Systems	21
	10.1 pgvector (Postgres extension) --- the "use what you have" choice	22
	10.2 The major systems	22
	10.3 Managed vs self-hosted	23
11	Scaling, Updates & Operations	23
	11.1 Sharding	23
	11.2 Replication	24
	11.3 Inserts vs deletes --- the hard part	24
	11.4 Filtered search at scale (recap with the scaling angle)	24
	11.5 Re-embedding when the model changes (full rebuild)	24
	11.6 Freshness	25
12	Evaluation	25

13	Designing Search over 100M Documents (Worked Example)	26
13.1	1. Clarify requirements	26
13.2	2. Back-of-envelope sizing	27
13.3	3. Embedding model	27
13.4	4. Chunking	27
13.5	5. Index choice + params	27
13.6	6. Sharding & replication	27
13.7	7. Hybrid + rerank pipeline	27
13.8	8. Latency & resource budget	28
13.9	9. Trade-offs to state aloud	28
14	Architecture / Diagrams	28
14.1	Full RAG Pipeline (ingest + query)	28
14.2	HNSW search	29
14.3	IVF-PQ retrieval	29
14.4	Sharded vector service	29
14.5	Vector vs lexical vs hybrid	29
15	Real-World Examples	29
15.1	Google Search / YouTube (ScaNN)	29
15.2	Meta Ads & FAISS	30
15.3	Spotify (Annoy)	30
15.4	Pinterest visual search (PinSage)	30
15.5	Perplexity / enterprise RAG (Glean, Notion AI, Microsoft 365 Copilot)	30
15.6	Semantic cache for LLMs	30
15.7	Near-duplicate detection / dedup	30
16	Real-Life Analogies	30
17	Memory Tricks / Mnemonics	31
18	Common Interview Questions	32
18.1	Q1: Why not just use exact kNN or a k-d tree for vector search?	32
18.2	Q2: Explain HNSW. Why is it the default?	32
18.3	Q3: How does IVF-PQ let you index a billion vectors?	32
18.4	Q4: Cosine vs dot product vs L2 --- which and why for text?	32
18.5	Q5: How do you choose chunk size, and what's the trade-off?	33
18.6	Q6: Walk me through filtered search. What's the gotcha?	33
18.7	Q7: Why hybrid search, and how do you combine the two rankings?	33
18.8	Q8: Explain retrieve-then-rerank. Why two stages?	33
18.9	Q9: You upgrade your embedding model. What happens operationally?	34
18.10	Q10: How do you measure whether retrieval is good?	34
18.11	Q11: Design semantic search over 100M documents with <100 ms p95.	34
18.12	Q12: How do deletes work in a vector index?	34
19	Senior-Level Discussion Points	35
19.1	Memory is usually the binding constraint, not CPU	35
19.2	Two-stage retrieval is universal	35
19.3	Embedding quality dominates index quality	35
19.4	Hybrid + rerank beats bigger models	35
19.5	Filtering correctness is a security boundary	35
19.6	The re-embed treadmill	35
19.7	Freshness vs rebuild cost	35
19.8	Quantization recall recovery	36
20	Typical Mistakes Candidates Make	36
21	How This Connects to Other Topics	36
22	FAANG Interview Tips	37
23	Revision Cheat Sheet	37

23.1 10-Minute Summary	37
23.2 Key Numbers to Know	38
23.3 Most Important Concepts for Interviews	38
23.4 Cheat-Sheet Summary Table	39

How to use this file: This is a deep, mechanism-level reference for FAANG SDE/ML interviews on vector search, embeddings, ANN indexes, and RAG retrieval. Read top to bottom once to build the mental model; then use the **Revision Cheat Sheet** and **Common Interview Questions** the night before. Every section pairs an explanation with real code, concrete numbers, and a trade-off. When an interviewer says "design semantic search over 100M docs," the **Worked Example** section is your script.

1 Overview --- What It Is

A **vector database** is a system that stores high-dimensional numeric vectors (**embeddings**) and answers the query "**which stored vectors are closest to this query vector?**" in milliseconds, even across hundreds of millions of vectors.

An **embedding** is a dense array of floating-point numbers — say 768 of them — that a neural network produces to represent the *meaning* of a piece of content (a sentence, a paragraph, an image, a product). Two pieces of content that mean similar things land near each other in this 768-dimensional space; unrelated content lands far apart. The vector database makes "find the nearest meanings" fast.

Layer	What it does	Examples
Embedding model	Turns text/image into a dense vector	text-embedding-3-small, bge-large, e5, CLIP
Distance metric	Defines "closeness" between vectors	cosine, dot product, L2 (Euclidean)
ANN index	Finds approximate nearest neighbors fast	HNSW, IVF, IVF-PQ, ScaNN, Annoy, LSH
Vector store	Persists vectors + metadata, serves queries	Pinecone, Weaviate, Milvus, Qdrant, pgvector
Retrieval pipeline	Chunk → embed → upsert → retrieve → rerank	RAG, semantic search, recommendations
Filtering	Restrict search by metadata (date, tenant, ACL)	"only docs from tenant X, after 2024"

Key insight: A vector database is not a replacement for Postgres. It is a specialized index — the moral equivalent of a B-tree, but for *semantic proximity* in high-dimensional space instead of *sorted order* on a scalar key. The hard engineering problem it solves is that **exact nearest-neighbor search in high dimensions is brutally slow**, so it trades a tiny amount of accuracy (recall) for orders-of-magnitude speedups.

```
"How do I reset my password?" -embed→ [0.12, -0.43, 0.87, ..., 0.31] (768 floats)
"Steps to recover account access" -embed→ [0.11, -0.41, 0.85, ..., 0.29] ← very close
"What is photosynthesis?" -embed→ [-0.67, 0.82, -0.34, ..., 0.91] ← far away
```

```
Vector DB job: given the first vector, return the second (and not the third)
                 from among 100,000,000 stored vectors in < 50 ms.
```

2 Why It Exists

The core problem: meaning is not keywords.

Traditional search (inverted index + BM25, the engine behind Elasticsearch and classic Google) matches **tokens**. If you search "reset password" it finds documents containing the literal words "reset" and "password." This breaks in three ways:

1. **Synonymy** — "recover account access" means the same thing but shares zero keywords with "reset password." Keyword search misses it entirely.
2. **Polysemy** — "Apple stock" (fruit inventory vs. AAPL shares) — the same token, different meanings. Keyword search can't tell them apart.
3. **Paraphrase / intent** — "my laptop won't turn on" should match a doc titled "troubleshooting power issues on portable computers." No shared tokens.

Embeddings solve this because the model maps *meaning* to *geometry*. "Reset password" and "recover account access" produce nearly identical vectors even though they share no words.

The second core problem: exact nearest-neighbor search is too slow.

Once you have vectors, you need to find the nearest ones. The naive approach — compute the distance from the query to every stored vector and sort — is **exact k-Nearest-Neighbor (kNN)**, and it costs $O(N \cdot d)$ per query.

```

1 N = 100,000,000 documents
2 d = 768 dimensions
3 Cost per query = N × d = 76.8 billion multiply-adds
4
5 At ~10 GFLOP/s effective for a brute-force dot-product loop,
6 that's ~7.6 seconds per query. Per query. Unusable.
```

You might think: "use a tree, like a k-d tree, the way we'd do 2D nearest-neighbor." That fails because of the **curse of dimensionality**:

2.1 Why k-d trees fail in high dimensions

A k-d tree partitions space by axis-aligned splits and prunes branches that can't contain a closer point. In 2D or 3D this prunes ~99% of the tree and gives $O(\log N)$ search. But as dimensions grow:

- ▶ **Volume concentrates in the corners.** The fraction of a hypercube's volume near the surface goes to 1. In 768 dimensions, essentially all points are "near the boundary," so the pruning bound rarely lets you skip a branch.
- ▶ **Distances concentrate.** The ratio of the farthest to the nearest point distance approaches 1 as $d \rightarrow \infty$. When the nearest and farthest neighbors are nearly equidistant, "nearest neighbor" is barely meaningful and pruning is impossible.
- ▶ **Branching factor explosion.** To prune effectively in d dimensions you'd need $\sim 2^d$ cells; for $d=768$ that's astronomically more than any dataset.

Exact NN search method	Works well up to dimension
k-d tree / ball tree	~10-20
Cover tree	~30-50
Brute force ($O(N \cdot d)$)	any d , but $O(N)$ per query — too slow at scale
ANN (HNSW/IVF/PQ)	hundreds to thousands ← the only practical answer

This is exactly why **Approximate** Nearest Neighbor (ANN) exists: give up the guarantee

of finding *the* exact nearest neighbor, accept finding *almost always the right ones* (e.g., 95–99% recall), and get a 100–1000× speedup. The “approximate” is the whole point.

Before vector databases: semantic search required hand-built NLP — TF-IDF, latent semantic analysis (LSA/SVD), query expansion with synonym dictionaries, and entity linking. Each was brittle and shallow.

After embeddings + ANN: a single embedding model plus an ANN index gives deep semantic matching out of the box, and it generalizes to images, audio, and code with the same machinery.

3 Why FAANG Cares

Google: ANN is foundational. **ScaNN** (Scalable Nearest Neighbors) is Google's own library, used in Search and YouTube recommendations. Vertex AI Matching Engine serves billion-vector indexes. Embedding-based retrieval is how Google does semantic ranking, “related searches,” and RAG grounding for Gemini. Expect questions on ScaNN's anisotropic quantization and on grounding LLMs.

Meta: Built **FAISS** (Facebook AI Similarity Search) — the most-used ANN library in the world. Meta runs embedding retrieval for feed ranking, ads retrieval (find candidate ads from billions), Reels recommendations, and content moderation (find near-duplicate / similar harmful content). “Two-tower” retrieval models feeding FAISS are a core interview topic.

Amazon: Product search and recommendations (“customers who bought this”) are embedding retrieval at planetary scale. OpenSearch (their Elasticsearch fork) ships kNN/HNSW. Amazon Bedrock Knowledge Bases is managed RAG. Cost-per-query and memory footprint directly hit AWS margins, so quantization (PQ) and sharding come up.

Microsoft: Azure AI Search has native vector + hybrid search; it underpins Microsoft 365 Copilot's retrieval over your org's documents. They care about hybrid (dense + BM25) and ACL-aware filtered search (you must only retrieve docs you're allowed to see).

Apple: On-device semantic search (Spotlight, Photos “find pictures of dogs”), privacy-first, tight memory budgets — making PQ/compression and small embedding models matter.

Netflix / Spotify / Pinterest: Recommendations are retrieval. Spotify built **Annoy** (Approximate Nearest Neighbors Oh Yeah) for music recs. Pinterest's PinSage/visual search is embedding retrieval over billions of pins.

Startups (Perplexity, Glean, Harvey, Notion AI, Cursor): The product *is* retrieval quality. Every one of them ships RAG; chunking, hybrid search, and reranking are daily decisions.

Why interviewers ask: RAG is now standard in every AI product, so retrieval quality is a core competency. The recall/latency/memory trade-off is a clean systems-design question with real numbers. And it connects ML (embeddings, training objectives) with systems (indexes, sharding) — a great signal for senior generalists.

4 Core Concepts

4.1 Semantic search vs. lexical search

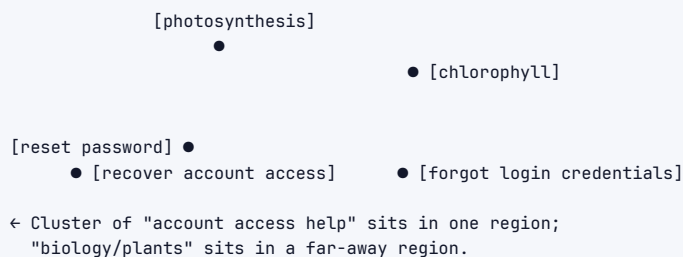
	Lexical (BM25 / inverted index)	Semantic (embeddings + ANN)
Matches on	Exact tokens / stems	Meaning / intent
Strength	Rare terms, IDs, names, acronyms, code	Paraphrase, synonyms, concepts
Weakness	Synonyms, paraphrase	Exact strings (“error code E-1042”), out-of-domain jargon

	Lexical (BM25 / inverted index)	Semantic (embeddings + ANN)
Index	Posting lists (sparse)	Dense vectors in ANN index
Scoring	TF-IDF / BM25	Cosine / dot product distance
Best in practice	—	Hybrid: use both, fuse the scores

The professional answer is almost never "semantic only." It's **hybrid search** — semantic for recall on meaning, lexical for precision on exact terms — fused together (covered later).

4.2 The vector space

After embedding, every document is a point in $\mathbb{R}^d$. The geometry encodes semantics:



Three operations matter:

- **Nearest-neighbor query** — given a query point, return the k closest stored points (search/RAG).
- **Clustering** — group points into regions (used internally by IVF; also for dedup, topic discovery).
- **Filtered NN** — nearest neighbors *subject to* a metadata predicate ("only points where tenant=X").

4.3 Exact kNN cost recap

```

1 import numpy as np
2
3 def exact_knn(query, vectors, k):
4     # vectors: (N, d) matrix, query: (d,)
5     # 0(N*d) dot products + 0(N log k) selection
6     sims = vectors @ query          # N*d multiply-adds
7     idx = np.argpartition(-sims, k)[:k]
8     return idx[np.argsort(-sims[idx])]
```

At $N=100M$, $d=768$ this is ~ 7.6 s/query (shown above). ANN indexes cut this to single-digit milliseconds by *not* looking at most vectors — they navigate a graph (HNSW) or only probe a few clusters (IVF).

Interview takeaway: Brute force is fine up to $\sim$ tens of thousands of vectors (a few ms). Past $\sim 100K$ – $1M$ you need an ANN index. State this threshold explicitly.

5 Embeddings

5.1 What a dense embedding is

A dense embedding is a fixed-length vector of real numbers where **every dimension carries (distributed) information** — unlike a sparse one-hot or bag-of-words vector where most entries are zero. "Dense" means all d entries are meaningful floats; "embedding" means the vector lives in a learned space where geometric closeness $\approx$ semantic similarity.

```
Sparse (bag-of-words, vocab=50,000): [0,0,...,1,...,0,1,0,...,0] ← 50,000 dims, mostly 0
Dense (embedding):                  [0.12, -0.43, 0.87, ..., 0.31] ← 768 dims, all informative
```

No single dimension means "is about cats." Meaning is **distributed** across all dimensions — direction in the space encodes concepts.

5.2 How models learn embeddings --- the training objective

Embeddings are learned, not designed. The dominant objective is **contrastive learning**: pull semantically-related pairs together and push unrelated pairs apart.

Contrastive / InfoNCE loss (the core idea):

For an anchor a , a positive p (something that should be similar), and negatives n_i (things that should be dissimilar), minimize:

$$\text{loss} = -\log\left(\frac{\exp(\text{sim}(a, p) / \tau)}{\sum_i \exp(\text{sim}(a, n_i) / \tau)} \right)$$

Maximizing the numerator pulls a and p together; the denominator pushes a away from all negatives. τ is a temperature. **In-batch negatives** are the trick that makes this cheap: within a batch of B (anchor, positive) pairs, every other pair's positive serves as a negative, giving $B-1$ negatives for free.

Self-supervised pretraining generates the pairs without human labels:

- › **Inverse cloze / span corruption** — a sentence and its surrounding paragraph are a positive pair.
- › **Dropout-as-augmentation (SimCSE)** — feed the *same* sentence through the encoder twice with different dropout masks; the two outputs are a positive pair. Brilliantly simple and strong.
- › **Title-body, question-answer, query-clicked-document** mined from logs are natural positives.

Sentence-transformers & bi-encoders: A **bi-encoder** encodes the query and the document *independently* into vectors, then compares with a cheap dot product. This is what makes retrieval scalable: you embed all N documents *once, offline*, store the vectors, and at query time only embed the query (one forward pass) and do ANN search. Contrast with a **cross-encoder** (used for reranking) which feeds [query, doc] *together* through attention — far more accurate but $O(N)$ forward passes, impossible at retrieval scale.

```
1 from sentence_transformers import SentenceTransformer
2
3 model = SentenceTransformer("BAAI/bge-base-en-v1.5") # bi-encoder, 768-dim
4
5 # Offline: embed the whole corpus once
6 doc_vecs = model.encode(documents, batch_size=256, normalize_embeddings=True)
7
8 # Online: embed only the query (one forward pass), then ANN search
9 q = model.encode("how do I reset my password", normalize_embeddings=True)
10 # similarity = dot product because vectors are unit-normalized
```

5.3 Dimensionality --- 384 vs 768 vs 1536 vs 3072

Dims	Typical models	Pros	Cons
384	all-MiniLM-L6-v2, bge-small	Tiny, fast, 4× less RAM than 1536	Lower ceiling on quality
768	bge-base, e5-base, nomic-embed	Sweet spot quality/cost	Standard default
1024	bge-large, cohere-embed-v3	Strong quality	More RAM/compute
1536	OpenAI text-embedding-3-small, ada-002	Excellent, managed API	2× RAM vs 768
3072	OpenAI text-embedding-3-large	Best OpenAI quality	4× RAM vs 768; often overkill

The trade-off is concrete. Memory and compute scale linearly with d:

```
100M vectors, float32 (4 bytes):
d=384 → 100M × 384 × 4 B = 153.6 GB
d=768 → 100M × 768 × 4 B = 307.2 GB
d=1536 → 100M × 1536 × 4 B = 614.4 GB
d=3072 → 100M × 3072 × 4 B = 1.23 TB
```

Doubling dimensions doubles RAM, doubles distance-computation cost, and doubles network/storage. Higher d usually improves recall — but with diminishing returns. **Matryoshka embeddings** (MRE, used by OpenAI v3 and nomic) train so that the *first* k dimensions are themselves a usable embedding — you can truncate 1536→512 and keep ~90%+ of quality, getting a runtime knob for the cost/quality trade-off.

5.4 Normalization --- unit vectors

Most text embeddings are **L2-normalized** to unit length ($\|v\| = 1$). This matters because:

- ▶ After normalization, **dot product = cosine similarity** (proof below), so you can use the faster inner product metric in the index and still get cosine ranking.
- ▶ It removes magnitude as a confound — you compare *direction* (meaning) only, not length.

```
1 v = v / np.linalg.norm(v) # now ||v|| = 1; dot(a,b) = cosine(a,b)
```

If your model emits normalized vectors, configure the index for **inner product (IP)**, not L2 — it's the same ranking and slightly cheaper.

5.5 Choosing / fine-tuning a domain model

Off-the-shelf models are trained on web/general text. For specialized domains (legal, biomedical, code, your internal jargon) they under-perform because the *geometry* doesn't separate your domain's concepts well.

Decision order:

1. **Start with a strong general model** (bge-large, e5, OpenAI v3). Measure recall on a golden set first.
2. **Try a domain pretrained model** if one exists (BioBERT/PubMedBERT for biomed, CodeBERT/jina-code for code).
3. **Fine-tune with contrastive pairs from your data** — mined (query, clicked-doc) pairs, or synthetic (LLM generates a question for each chunk → (question, chunk) positive). A few thousand pairs can lift recall@10 by 5–15 points.

```

1 from sentence_transformers import SentenceTransformer, losses, InputExample
2 from torch.utils.data import DataLoader
3
4 model = SentenceTransformer("BAAI/bge-base-en-v1.5")
5 train = [InputExample(texts=[q, positive_chunk]) for q, positive_chunk in mined_pairs]
6 loader = DataLoader(train, shuffle=True, batch_size=64)
7 loss = losses.MultipleNegativesRankingLoss(model) # in-batch negatives
8 model.fit(train_objectives=[(loader, loss)], epochs=1, warmup_steps=100)

```

The hidden cost: fine-tuning changes the space. New vectors are *not comparable* to old ones, so you must **re-embed your entire corpus** with the new model. At 100M chunks and 1000 chunks/sec/GPU that's ~28 GPU-hours per pass — a real operational event.

5.6 Multimodal --- CLIP and shared spaces

CLIP (Contrastive Language–Image Pretraining) trains an image encoder and a text encoder *jointly* with a contrastive loss over (image, caption) pairs, so an image and its caption land at the **same point** in a shared 512/768-dim space. This enables cross-modal search: embed a text query and retrieve images, or embed an image and retrieve captions, all with one ANN index.

```

1 # Search images with a text query — one shared space
2 image_vecs = clip.encode_image(images) # stored offline
3 q = clip.encode_text("a golden retriever on a beach")
4 hits = index.search(q, k=10) # returns nearest IMAGE vectors

```

The same recipe gives audio–text (CLAP), video–text, and document–layout embeddings. **Interview point:** multimodal RAG over PDFs with figures uses this — embed page images and surrounding text into one space.

5.7 Embedding drift & the cost of re-embedding

Two distinct problems share the name "drift":

1. **Model drift (you change models):** A new model version → entirely new geometry → **full corpus re-embedding required** + index rebuild. Plan blue/green: build the new index alongside the old, validate recall on the golden set, then cut over. Never mix vectors from two models in one index.
2. **Data/semantic drift (the world changes):** New slang, new products, new entities appear that the embedding model (trained at an earlier cutoff) represents poorly. Symptoms: recall degrades over time on fresh queries. Mitigations: periodic fine-tuning, hybrid search (BM25 catches new exact terms the model never saw), and monitoring recall on a rolling golden set.

```

1 Re-embed cost model (100M chunks):
2   throughput  = 1,000 chunks/sec/GPU (bge-base on an A10)
3   total time  = 100,000,000 / 1,000 = 100,000 GPU-sec ≈ 27.8 GPU-hours
4   with 8 GPUs ≈ 3.5 wall-clock hours + index build
5   API cost (if using OpenAI v3-small @ ~$0.02/1M tokens, ~300 tok/chunk)
6   ≈ 100M × 300 / 1M × $0.02 = $600 per full re-embed

```

Interview takeaway: Re-embedding is the single biggest hidden operational cost of a vec-

tor system. Always mention it when someone proposes "just upgrade the embedding model."

6 Similarity & Distance Metrics

The index needs a notion of "closeness." Three matter; one dominates text.

6.1 Cosine similarity

Measures the **angle** between two vectors, ignoring magnitude.

```
1 cos(a, b) = (a · b) / (||a|| · ||b||)
2           = Σ a_i b_i / (sqrt(Σ a_i^2) · sqrt(Σ b_i^2))
3
4 Range: [-1, 1] (1 = identical direction, 0 = orthogonal, -1 = opposite)
5 Cosine distance = 1 - cos(a, b)
```

Why cosine is the text default: Document length / token count inflates raw vector magnitude, but meaning is in the *direction*. Two paraphrases of different lengths should match — cosine ignores the length difference and compares semantic direction. Almost every sentence-transformer is trained with cosine objectives, so the geometry is calibrated for it.

6.2 Dot product (inner product)

```
1 a · b = Σ a_i b_i = ||a|| ||b|| cos(a, b)
```

Cosine **scaled by both magnitudes**. Use it when magnitude is meaningful (e.g., recommendation models that bake "popularity" or "confidence" into the norm). MIPS = Maximum Inner Product Search.

6.3 The normalized → cosine = dot equivalence (know this cold)

If both vectors are unit length ($\|a\| = \|b\| = 1$):

$$a \cdot b = \|a\| \|b\| \cos(a, b) = 1 \cdot 1 \cdot \cos(a, b) = \cos(a, b)$$

So: **normalize once, then use the cheap dot-product metric, and you get exact cosine ranking**. This is why production systems L2-normalize at embed time and configure the index for inner product. It saves the per-query norm computation.

6.4 Euclidean / L2 distance

$$L2(a, b) = \sqrt{\sum (a_i - b_i)^2}$$

Straight-line distance. For **unit-normalized** vectors, L2 and cosine produce the *same ranking* (they're monotonically related):

$$\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2(a \cdot b) = 1 + 1 - 2 \cos(a, b) = 2 - 2 \cos(a, b)$$

→ smaller L2 ⇔ larger cosine. Same nearest neighbors.

So on normalized text vectors, cosine / dot / L2 all rank identically — pick whichever your

DB optimizes. On *un-normalized* vectors they differ, and L2 is sensitive to magnitude.

6.5 Manhattan (L1)

$$L1(a, b) = \sum |a_i - b_i|$$

Sum of absolute differences. Rare in semantic search (less aligned with how embeddings are trained), more common in some recommender and tabular settings. Mention it for completeness; don't default to it for text.

6.6 Which metric, when

Metric	Use when	Notes
Cosine	Text embeddings (default)	Direction = meaning; length-invariant
Dot / IP	Normalized vectors (= cosine, cheaper) OR magnitude encodes signal (recsys)	MIPS; OpenAI/Cohere vectors normalized
L2 (Euclidean)	Image embeddings, normalized text (same ranking as cosine)	FAISS default metric
L1 (Manhattan)	Some tabular / recsys cases	Uncommon for text

Interview takeaway: For text, say "cosine — but since I normalize, I configure the index for inner product, which gives identical ranking with less compute." That one sentence signals you understand the equivalence.

7 Approximate Nearest Neighbor (ANN) Indexes

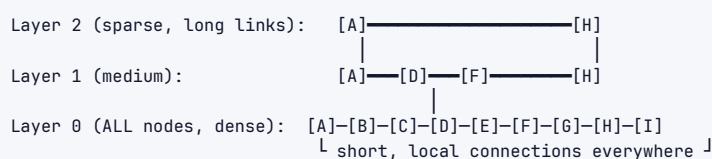
The heart of the system. Each index makes a different bet about how to avoid scanning all N vectors. Know HNSW deeply; know IVF and PQ well enough to combine them.

7.1 HNSW --- Hierarchical Navigable Small World (the default)

Intuition: skip lists + small-world graphs. A skip list speeds up linked-list search by adding sparse "express lanes" on top. HNSW does the same for nearest-neighbor search: it builds a multi-layer graph where the top layers have few nodes with long-range links (express lanes) and the bottom layer has every node with short-range links (local roads). You enter at the top, take big hops to get near the answer, then drop down to refine.

A **navigable small-world (NSW)** graph is one where greedy "always move to the neighbor closest to the query" reaches the true nearest neighbor in a few hops — like the "six degrees of separation" property of social networks. HNSW makes it *hierarchical* for $O(\log N)$ navigation.

HNSW layered structure (query q enters at the top):



Greedy search for q:

1. Enter at A (top layer). Move to whichever neighbor is closest to q.

- ```

A → H (H is closer to q at layer 2). No closer neighbor → descend.
2. Layer 1 from H: check H's neighbors (F, ...). F closer → move to F.
 No closer → descend.
3. Layer 0 from F: explore E, G, ... maintain a candidate set of size ef_search,
 greedily expand the closest unvisited, stop when no improvement.
4. Return the top-k from the candidate set.

```

Total hops  $\approx O(\log N)$ . Each hop touches a handful of neighbors.

**Greedy search walkthrough (concrete):** Suppose  $q$  is closest to node  $E$ . At layer 2 we hop  $A \rightarrow H$  (overshoot, but cheap). At layer 1 we hop  $H \rightarrow F$  (getting warmer). At layer 0 we explore  $F$ 's neighborhood, keep the `ef_search` best candidates in a priority queue, expand the closest, discover  $E$ , find nothing closer, and return  $E$  (plus the next-best as the rest of top- $k$ ). We touched maybe 50–200 nodes out of 100M.

**Construction.** Insert nodes one at a time. Each new node picks a random maximum layer (geometric distribution — most nodes only exist at layer 0, few reach the top). For each layer from its top down to 0, it runs the same greedy search to find its  $M$  nearest existing nodes and links to them. `ef_construction` controls how thorough that search is during building.

#### Parameters:

| Param               | Meaning                                 | Higher value →                           | Typical                         |
|---------------------|-----------------------------------------|------------------------------------------|---------------------------------|
| <b>M</b>            | Max neighbors per node (graph degree)   | Better recall, more memory, slower build | 16–48                           |
| <b>ef_construct</b> | Candidate list size during <i>build</i> | Better graph quality, slower build       | 100–400                         |
| <b>ef_search</b>    | Candidate list size during <i>query</i> | Higher recall, higher latency            | 50–500 (tunable at query time!) |

The killer feature: **ef\_search is a query-time knob**. You build the graph once, then dial recall vs. latency per query without rebuilding — set `ef_search=64` for a fast path and `ef_search=512` for a high-recall path.

**Memory cost.** HNSW stores the full vectors *plus* the graph edges:

```

Per vector: d × 4 bytes (float32) + M × ~8 bytes (neighbor links, both directions ~2M)
For d=768, M=32:
 vector = 768 × 4 = 3,072 bytes
 graph ≈ 2 × 32 × 8 = 512 bytes
 total ≈ 3,584 bytes/vector
100M vectors ≈ 358 GB RAM. ← This is why HNSW alone doesn't scale to billions cheaply.

```

**Why it's the default:** highest recall at a given latency, no training step, supports incremental inserts, query-time recall knob. Used by Pinecone, Weaviate, Qdrant, Milvus, pgvector, Elasticsearch/OpenSearch, Lucene. **Cons:** memory-hungry (stores full vectors), deletes are awkward (tombstones; graph degrades, eventually rebuild), build is slower than IVF.

## 7.2 IVF --- Inverted File (cluster-and-probe)

**Intuition:** Partition the space into clusters once; at query time, only search the few clusters near the query instead of all  $N$  vectors.

#### Mechanism (Voronoi cells via k-means):

1. **Train** a coarse quantizer: run k-means on a sample to find  $n_{list}$  centroids (e.g., 4096). Each centroid "owns" a Voronoi cell.
2. **Assign** every vector to its nearest centroid → an inverted list per centroid (centroid → [vector ids in that cell]).

3. **Query:** find the `nprobe` centroids nearest the query, then brute-force search only the vectors in those `nprobe` lists.

Space partitioned into `nlist=8` Voronoi cells (centroids ●):



`q` = query  
`nprobe=2` → search cells C5 and C6 only  
 (the two centroids nearest `q`),  
 skip the other 6 cells entirely.

### Parameters:

| Param         | Meaning                        | Trade-off                                                                                                                                                     |
|---------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>nlist</b>  | Number of clusters (centroids) | More cells = fewer vectors/cell = faster probe, but need higher <code>nprobe</code> for recall. Rule of thumb: $nlist \approx \sqrt{N}$ to $4 \cdot \sqrt{N}$ |
| <b>nprobe</b> | Cells searched per query       | Higher = better recall, slower. Query-time knob, like <code>ef_search</code>                                                                                  |

`N` = 100M, `nlist` = 16,384 → ~6,100 vectors per cell on average  
`nprobe` = 32 → search  $\sim 32 \times 6,100 = \sim 195,000$  vectors (vs 100M brute force)  
 → ~500× fewer distance computations

**Training requirement (the catch):** IVF must be *trained* (k-means) on a representative sample before you can add vectors. If your data distribution shifts, the cells become unbalanced (some huge, some empty) and recall drops — you must retrain. HNSW needs no training. The **edge-of-cell problem**: a query near a cell boundary may have its true nearest neighbor in an adjacent, un-probed cell — that's a recall miss, mitigated by higher `nprobe`.

**When to use:** very large datasets where HNSW memory is too high, and you can tolerate a training step. Almost always combined with **PQ** (next) to cut memory.

## 7.3 Product Quantization (PQ) --- compressing vectors

**The problem PQ solves:** storing full float32 vectors costs too much RAM at billion scale (a  $1B \times 768 \times 4B$  index = 3 TB). PQ compresses each vector to a few bytes.

### Mechanism:

1. **Split** each `d`-dim vector into `m` sub-vectors (e.g., `d=768`, `m=96` → each sub-vector is 8 dims).
2. For each of the `m` sub-spaces, run k-means to learn a **codebook** of `k=256` centroids (so each centroid id fits in 1 byte).
3. **Encode** a vector by replacing each sub-vector with the *id* of its nearest codebook centroid → the whole vector becomes `m` bytes.

Original vector (768 float32 = 3,072 bytes):  
`[-sub1-][-sub2-]...[-sub96-]` each sub = 8 floats

After PQ (`m=96` sub-vectors, 256 centroids each):  
`[id1][id2]...[id96]` each id = 1 byte → 96 bytes total

Compression: 3,072 → 96 bytes = 32× smaller.

**Asymmetric Distance Computation (ADC):** at query time you do *not* quantize the query. Instead, you precompute a lookup table: for each sub-space, the distance from the query's sub-vector to all 256 centroids (`m` × 256 distances). Then the approximate distance to any stored code is just `m` table lookups + adds — no full distance math. This keeps accuracy higher than quantizing both sides ("symmetric").



```

1 PQ distance(query, code):
2 precompute LUT[m][256] = dist(query_sub_j, centroid_c) for all j, c # once per query
3 for each stored vector with codes [c1..cm]:
4 approx_dist = LUT[0][c1] + LUT[1][c2] + ... + LUT[m-1][cm] # m adds

```

### The memory math (the number to quote):

```

1 BILLION vectors, d=768:
float32 (no compression): 1e9 × 768 × 4 B = 3.07 TB (won't fit in RAM)
PQ, m=96, 8-bit codes: 1e9 × 96 × 1 B = 96 GB (fits on one big box)
→ 32× memory reduction.
Cost: recall drops (lossy compression). Typically recall@10 ~0.85-0.95
 depending on m, recoverable by re-ranking the PQ candidates with exact
 distances on the float vectors (if kept on disk/SSD).

```

**IVF-PQ (the workhorse for billion-scale):** combine them — IVF narrows to nprobe cells (avoid scanning all N), PQ compresses the vectors in those cells (fit in RAM). This is the standard FAISS recipe for billion-vector indexes (IndexIVFPQ). A refinement, **IVF-PQ + reranking** (IndexIVFPQR / 0PQ): retrieve top-100 by PQ approx distance, then re-score those 100 with exact float distances for final ranking.

```

1 import faiss
2 d, nlist, m, nbits = 768, 16384, 96, 8
3 quantizer = faiss.IndexFlatIP(d) # coarse quantizer (centroids)
4 index = faiss.IndexIVFPQ(quantizer, d, nlist, m, nbits)
5 index.train(sample_vectors) # k-means for IVF + PQ codebooks
6 index.add(all_vectors) # stored as PQ codes
7 index.nprobe = 32
8 D, I = index.search(query, k=10)

```

## 7.4 LSH --- Locality-Sensitive Hashing

**Intuition:** design hash functions so that *similar* vectors collide (land in the same bucket) with high probability. Query by hashing the query and only scanning its bucket(s).

For cosine, use **random hyperplane hashing**: pick  $r$  random hyperplanes; each vector's hash bit is the sign of its dot product with that hyperplane's normal. Vectors on the same side of all hyperplanes share a hash → likely similar.

```

hash(v) = (sign(v·r1), sign(v·r2), ..., sign(v·rb)) # b bits, b random hyperplanes
Similar vectors → same signs → same bucket → small candidate set to brute-force.
Multiple hash tables (L of them) raise recall (a miss in one table may hit another).

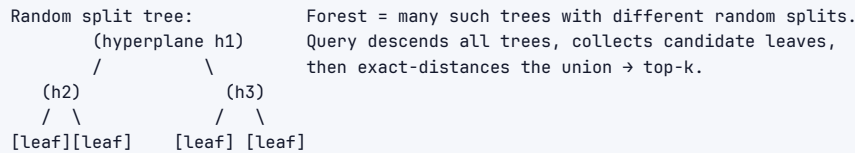
```

**Trade-off:** simple, theoretically clean, supports streaming inserts. But in practice it needs many tables/bits to hit high recall, which costs memory, and it generally loses to HNSW on the recall-per-byte and recall-per-millisecond curves. Mostly of historical/theoretical importance now; still used for near-duplicate detection (MinHash for sets).

## 7.5 Tree-based --- Annoy

**Annoy** (Spotify) builds a **forest of random projection trees**. Each tree recursively splits the dataset by a random hyperplane; a leaf holds a small bucket. Querying descends multiple trees and unions the leaf candidates.





**Trade-off:** memory-mapped files → great for **read-only**, shareable indexes (Spotify serves the same index across many machines via mmap). But the index is **immutable** — you rebuild to add data. Lower recall-per-memory than HNSW. Good when you have a static corpus and want simple, mmap-able files.

## 7.6 ScaNN --- Anisotropic quantization (Google)

**ScaNN's insight:** standard PQ minimizes *reconstruction* error (how well the code approximates the original vector). But for **maximum inner product search** what actually matters is preserving the *inner product with likely query directions*. ScaNN uses **anisotropic vector quantization** — it weights quantization error by how much it distorts inner products in the directions that matter, penalizing errors parallel to the vector more than orthogonal ones. Result: higher recall at the same compression. Combined with a fast SIMD-optimized scan, it tops the ann-benchmarks recall/latency frontier for in-memory MIPS.

**Trade-off:** state-of-the-art speed/recall, but more complex to tune; strongest as a library (or via Vertex AI) rather than a turnkey DB.

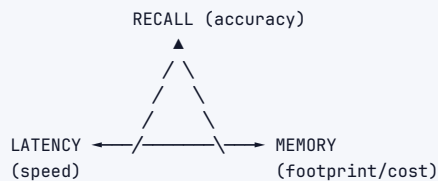
## 7.7 ANN comparison table

| Index             | Build time | Query speed      | Recall                       | Memory                     | Inserts                       | Training?                 | Best for                         |
|-------------------|------------|------------------|------------------------------|----------------------------|-------------------------------|---------------------------|----------------------------------|
| <b>HNSW</b>       | Slow       | Very fast        | Very high                    | High (full vecs + graph)   | Incremental                   | No                        | Default; <~100M, recall-critical |
| <b>IVF (Flat)</b> | Fast       | Fast             | High (tune nprobe)           | Full vecs                  | Add anytime; retrain on drift | Yes (k-means)             | Large, memory-ok                 |
| <b>IVF-PQ</b>     | Moderate   | Fast             | Moderate-high                | <b>Very low</b> (PQ codes) | Add anytime                   | Yes (k-means + codebooks) | Billion-scale, RAM-bound         |
| <b>PQ (flat)</b>  | Moderate   | Fast             | Moderate                     | Very low                   | Add anytime                   | Yes (codebooks)           | Memory-extreme                   |
| <b>LSH</b>        | Fast       | Fast             | Moderate (needs many tables) | Moderate-high              | Streaming                     | No                        | Near-dup detection, theory       |
| <b>Annoy</b>      | Moderate   | Fast             | Moderate                     | Low (mmap)                 | <b>Immutable</b> (rebuild)    | No                        | Static, read-only, mmap-shared   |
| <b>ScaNN</b>      | Moderate   | <b>Very fast</b> | Very high                    | Low-moderate               | Rebuild-ish                   | Yes                       | In-memory MIPS at scale (Google) |

**Interview takeaway:** Default to **HNSW** for most workloads (<100M, recall matters, RAM available). Reach for **IVF-PQ** when memory is the binding constraint (billions of vectors). Mention **ScaNN** if asked about Google or the absolute speed/recall frontier. Mention **Annoy** for static, mmap-shared, read-only indexes.

## 8 The Recall / Latency / Memory Trade-off

Every ANN system lives inside a triangle. You cannot maximize all three; you pick a point.



HNSW with high M, high ef\_search → high recall, high memory, moderate latency  
 IVF-PQ with low m, low nprobe → low memory, low latency, lower recall  
 Brute force → perfect recall, huge latency, full memory

### 8.1 Recall@k --- the metric

Recall@k = of the *true* top-k nearest neighbors (per exact brute force), what fraction did the ANN index return in *its* top-k?

True top-10 for a query: {d3, d7, d9, d11, d14, d20, d22, d31, d40, d55}  
 ANN returned top-10: {d3, d7, d9, d11, d14, d20, d22, d31, d40, d99}  
 9 of 10 correct → recall@10 = 0.90

You compute it by running exact kNN on a sample of queries (the ground truth) and comparing. Target recall depends on the product: web search tolerates ~0.90; medical/legal retrieval may demand ~0.99.

### 8.2 How parameters tune the triangle

| Want      | HNSW lever                                  | IVF / IVF-PQ lever                                   | Side effect               |
|-----------|---------------------------------------------|------------------------------------------------------|---------------------------|
| ↑ Recall  | ↑ ef_search, ↑ M                            | ↑ nprobe, ↑ nlist resolution, ↓ PQ compression (↑ m) | ↑ latency and/or ↑ memory |
| ↓ Latency | ↓ ef_search                                 | ↓ nprobe                                             | ↓ recall                  |
| ↓ Memory  | (limited — vecs are full)<br>use PQ variant | use PQ (↓ m bytes), 8-bit→4-bit                      | ↓ recall                  |

#### Concrete tuning scenario (100M, d=768, HNSW):

M=16, ef\_search=64 → recall@10 ≈ 0.92, p95 latency ≈ 8 ms, ~330 GB RAM  
 M=32, ef\_search=128 → recall@10 ≈ 0.97, p95 latency ≈ 15 ms, ~358 GB RAM  
 M=48, ef\_search=512 → recall@10 ≈ 0.99, p95 latency ≈ 40 ms, ~390 GB RAM

#### Concrete tuning scenario (1B, d=768, IVF-PQ):

m=96 (8-bit), nprobe=16 → recall@10 ≈ 0.82, p95 ≈ 6 ms, ~96 GB RAM  
 m=96, nprobe=64 → recall@10 ≈ 0.90, p95 ≈ 18 ms, ~96 GB RAM  
 m=96, nprobe=64 + rerank top-200 with float vecs on SSD → recall@10 ≈ 0.96, p95 ≈ 30 ms

**The reranking trick to break the triangle:** retrieve a *larger* candidate set cheaply (high speed, lower per-candidate accuracy), then re-score the small candidate set with exact distances (or a cross-encoder). You buy back recall/precision without paying full-index cost.

**Interview takeaway:** Never say "I'll use HNSW" and stop. Say: "HNSW with M=32, and I'd tune ef\_search against a recall@10 target of 0.97 measured on a golden set, accepting ~15 ms

p95 and ~358 GB RAM — if memory is the constraint I'd switch to IVF-PQ and rerank.” That's the senior answer.

## 9 Building a RAG / Semantic-Search Pipeline

RAG = Retrieval-Augmented Generation: retrieve relevant chunks from a vector store, inject them into an LLM prompt, generate a grounded answer. Semantic search is the same pipeline minus the generation step. Retrieval quality dominates final quality — “garbage retrieved, garbage generated.”

```
INGEST (offline): Docs → Parse → Chunk → Embed → Upsert(vector + metadata)
QUERY (online): Query → Embed → ANN retrieve top-k (+ BM25) → Fuse → Rerank → top 3-5 → LLM
```

### 9.1 Chunking strategies

You don't embed whole documents — you embed **chunks**, because (a) embedding models have token limits (256–512 tokens is the trained sweet spot for many), and (b) retrieval precision needs the *relevant passage*, not a 50-page PDF.

| Strategy                          | How                                                                                                 | Best for               | Pitfall                         |
|-----------------------------------|-----------------------------------------------------------------------------------------------------|------------------------|---------------------------------|
| <b>Fixed-size</b>                 | Every N tokens (e.g., 512)                                                                          | Quick start            | Cuts mid-sentence, splits ideas |
| <b>Recursive character</b>        | Split on <code>\n\n</code> , then <code>\n</code> , then <code>.</code> , then space until $\leq N$ | General text (default) | Still misses semantic units     |
| <b>Sentence / paragraph</b>       | Split on sentence/para boundaries                                                                   | FAQs, articles         | Uneven sizes                    |
| <b>Semantic chunking</b>          | Embed sentences; split where consecutive similarity drops                                           | Dense technical docs   | Expensive (embed to chunk)      |
| <b>Structural (Markdown/HTML)</b> | Split on headers H1/H2/H3, code blocks, tables                                                      | Docs, wikis            | Uneven chunk sizes              |
| <b>Parent-child</b>               | Retrieve small child chunk, feed larger parent to LLM                                               | Precision + context    | Two-level indexing              |

**Overlap.** Adjacent chunks share a sliding window (e.g., 50–100 tokens) so a sentence split across a boundary still appears whole in at least one chunk. Costs storage (more chunks) but prevents “answer cut in half” misses.

#### The size trade-off (with examples):

```
Chunk too SMALL (e.g., 64 tokens):
 Chunk = "Click Settings, then Security."
 Query = "how do I enable two-factor authentication?"
 → retrieved, but lacks the surrounding steps → LLM can't answer fully. (low context)

Chunk too LARGE (e.g., 2000 tokens):
 Chunk = entire 8-page "Account Security" article.
 Query = "reset password" → chunk matches, but 95% is irrelevant text that
 dilutes the embedding (averaged meaning) AND wastes the LLM context window.
 The relevant 2 sentences are buried. (low precision, high cost)

Sweet spot: 256-512 tokens, 10-20% overlap.
```

```

1 from langchain_text_splitters import RecursiveCharacterTextSplitter
2
3 splitter = RecursiveCharacterTextSplitter(
4 chunk_size=512, chunk_overlap=64, # ~512 tokens, ~12% overlap
5 separators=["\n\n", "\n", ". ", " ", ""], # prefer semantic boundaries
6)
7 chunks = splitter.split_text(document_text)

```

**Metadata is non-negotiable** — store it alongside every vector for filtering and citation:

```

1 record = {
2 "id": "doc42#chunk7",
3 "vector": embed(chunk_text),
4 "metadata": {
5 "text": chunk_text,
6 "source": "security_guide_v3.pdf",
7 "page": 12, "section": "Two-Factor Auth",
8 "tenant_id": "acme", "acl": ["support", "admin"],
9 "created_at": "2025-02-01", "doc_id": "doc42",
10 },
11 }

```

## 9.2 Embed + upsert

Use the **same model** for ingest and query (different models → incompatible geometry → garbage retrieval). Normalize. Upsert in batches.

```

1 import itertools
2 def batched(it, n):
3 it = iter(it)
4 while batch := list(itertools.islice(it, n)):
5 yield batch
6
7 for batch in batched(chunks, 256):
8 vecs = model.encode([c.text for c in batch], normalize_embeddings=True)
9 index.upsert([(c.id, v, c.metadata) for c, v in zip(batch, vecs)])

```

## 9.3 Retrieval top-k

Retrieve more than you'll show the LLM (e.g., top-20–50), then narrow via fusion/rerank. Over-retrieving improves the chance the right chunk is in the candidate set.

```

1 q = model.encode(user_query, normalize_embeddings=True)
2 candidates = index.search(q, k=30, filter={"tenant_id": "acme"})

```

## 9.4 Metadata filtering --- pre- vs post-filter (a real correctness/perf trap)

You often must restrict results ("only this tenant," "only docs the user can see," "only after 2024"). There are two ways, and the difference is a classic interview gotcha:

**POST-FILTER:** ANN returns top-k by vector distance, THEN drop those failing the predicate.  
 Problem: if only 1% of vectors match the filter, your top-30 might contain ZERO matches  
 → you return nothing, even though good matches exist deeper in the ranking.  
 Correctness bug. To compensate you over-fetch (k=3000), which is slow.

**PRE-FILTER:** Restrict the candidate set to matching vectors FIRST, then do ANN within it.  
 Correct results, but naive pre-filtering breaks the ANN graph (HNSW navigates through nodes that may be filtered out → can't traverse → recall collapses).

**PRODUCTION ANSWER:** "filtered ANN" – the index integrates the predicate INTO traversal.  
 Qdrant: payload index + filterable HNSW. Weaviate/Milvus: similar. pgvector: combine with a B-tree/partial index, or partition by tenant. For very selective filters (a handful of matches), fall back to exact brute force over the matching subset – it's tiny.

```

1 # Qdrant filtered search – predicate is applied DURING graph traversal
2 from qdrant_client.models import Filter, FieldCondition, MatchValue
3 hits = client.search(
4 collection_name="docs",
5 query_vector=q,
6 query_filter=Filter(must=[FieldCondition(key="tenant_id", match=MatchValue(value="acme"))]),
7 limit=30,
8)

```

**Interview takeaway:** When asked "how do you restrict search to a tenant?" the wrong answer is "filter the results afterward." The right answer names the post-filter recall bug and proposes filtered-ANN (predicate fused into traversal) or partitioning, with exact brute force for highly selective filters.

## 9.5 Hybrid search --- dense + sparse (BM25) with RRF

Dense embeddings miss exact strings (SKUs, error codes, rare names); BM25 nails them. BM25 misses paraphrase; dense nails it. Run **both** and fuse.

**Reciprocal Rank Fusion (RRF)** combines ranked lists using only the *rank* (not the raw, incomparable scores):

```
RRF_score(doc) = Σover_lists 1 / (k + rank_in_that_list) # k ≈ 60 by convention
```

A doc ranked #1 by BM25 and #3 by dense:  $1/(60+1) + 1/(60+3) = 0.0164 + 0.0159 = 0.0323$   
 A doc ranked #2 by dense only:  $1/(60+2) = 0.0161$   
 → The doc both methods like ranks highest. Robust, no score normalization needed.

```

1 def rrf(rank_lists, k=60, top_n=20):
2 scores = {}
3 for ranked in rank_lists: # each is [doc_id, ...] best-first
4 for rank, doc_id in enumerate(ranked):
5 scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
6 return sorted(scores, key=scores.get, reverse=True)[:top_n]
7
8 dense_hits = [h.id for h in index.search(q, k=50)]
9 bm25_hits = [h.id for h in bm25.search(user_query, k=50)]
10 fused = rrf([dense_hits, bm25_hits])

```

## 9.6 Reranking --- the two-stage retrieve → rerank pattern

Retrieval uses a **bi-encoder** (query and doc embedded *independently*) – fast, scalable, but it never lets the query and document "see" each other, so it's coarse. A **cross-encoder** feeds

[query, doc] *together* through a transformer with full cross-attention and outputs a single relevance score — far more accurate, but  $O(\text{candidates})$  forward passes, so you can't run it over the whole corpus.

**The two-stage architecture resolves this:** retrieve top-30–50 cheaply with the bi-encoder ANN index, then **rerank just those 30–50** with the cross-encoder and keep the top 3–5 for the LLM.

Stage 1 (recall): bi-encoder ANN over 100M docs → top 50 candidates (fast, coarse)  
 Stage 2 (precision): cross-encoder scores 50 (query,doc) pairs → top 5 (slow, accurate)  
 50 forward passes is cheap; 100M would be impossible.

```

1 from sentence_transformers import CrossEncoder
2 reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
3
4 def retrieve_and_rerank(query, top_n=5):
5 q = model.encode(query, normalize_embeddings=True)
6 candidates = index.search(q, k=50) # bi-encoder, fast
7 pairs = [(query, c.metadata["text"]) for c in candidates]
8 scores = reranker.predict(pairs) # cross-encoder, accurate
9 ranked = [c for _, c in sorted(zip(scores, candidates), key=lambda x: -x[0])]
10 return ranked[:top_n]

```

Commercial rerankers: **Cohere Rerank**, **Voyage rerank**, **Jina Reranker**. Reranking 50 docs adds ~30–80 ms but lifts answer quality substantially — almost always worth it in production.

## 9.7 Putting it together --- query pipeline

```

1 def rag_answer(user_query, tenant_id, user_acl):
2 q = model.encode(user_query, normalize_embeddings=True)
3 dense = index.search(q, k=50, filter={"tenant_id": tenant_id, "acl": {"$in": user_acl}})
4 bm25 = bm25_index.search(user_query, k=50, filter={"tenant_id": tenant_id})
5 fused_ids = rrf([h.id for h in dense], [h.id for h in bm25], top_n=50)
6 fused = [lookup(i) for i in fused_ids]
7 top = rerank(user_query, fused, top_n=5)
8 context = "\n\n".join(
9 f"[{c.metadata['source']}] p[{c.metadata['page']}] \n{c.metadata['text']}" for c in top
10)
11 return llm.generate(
12 f"Answer using ONLY this context. Cite sources.\n\n{context}\n\nQ: {user_query}"
13)

```

## 10 Vector Database Systems

You can use a **library** (FAISS, ScaNN, Annoy — you manage storage, sharding, persistence, filtering yourself) or a **database** (Pinecone, Weaviate, Milvus, Qdrant, pgvector — they add meta-data, filtering, replication, APIs, persistence).

## 10.1 pgvector (Postgres extension) --- the "use what you have" choice

If you're already on Postgres, pgvector adds a vector column type plus HNSW and IVF indexes. You get SQL joins, transactions, existing backups/replication, and ACID filtering *for free*.

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2
3 CREATE TABLE chunks (
4 id bigserial PRIMARY KEY,
5 doc_id text,
6 tenant_id text,
7 page int,
8 text text,
9 embedding vector(768) -- 768-dim dense vector
10);
11
12 -- HNSW index (cosine). vector_ip_ops for inner product, vector_l2_ops for L2.
13 CREATE INDEX ON chunks
14 USING hnsw (embedding vector_cosine_ops)
15 WITH (m = 16, ef_construction = 64);
16
17 -- Partial index for a tenant (helps filtered search)
18 CREATE INDEX ON chunks (tenant_id);
19
20 -- Query: top-10 nearest, filtered by tenant (pre-filter via WHERE + index)
21 SET hnsw.ef_search = 100; -- query-time recall knob
22 SELECT id, doc_id, page, text,
23 1 - (embedding <=> $1) AS cosine_similarity -- <=> is cosine distance
24 FROM chunks
25 WHERE tenant_id = 'acme'
26 ORDER BY embedding <=> $1 -- <=> cosine, <#> inner product, <-> L2
27 LIMIT 10;

```

$\Leftrightarrow$  = cosine distance,  $<\#>$  = negative inner product,  $\leftrightarrow$  = L2. **Caveat:** at very high N (>50–100M) and high-selectivity filters, pgvector's HNSW can lose recall under tight WHERE filters; partition by tenant or move to a dedicated vector DB.

## 10.2 The major systems

| DB              | Type         | Index                      | Scale           | Filtering                                  | Highlights                          | Best for                     |
|-----------------|--------------|----------------------------|-----------------|--------------------------------------------|-------------------------------------|------------------------------|
| <b>pgvector</b> | Postgres ext | HNSW, IVFFlat              | ~10s of M       | SQL WHERE (pre-filter)                     | SQL joins, ACID, reuse infra        | Already on Postgres          |
| <b>Pinecone</b> | Manage SaaS  | proprietary (HNSW-like)    | Billion         | Metadata filters                           | Zero ops, auto-scale, serverless    | Fast to production           |
| <b>Weaviate</b> | OSS + Cloud  | HNSW                       | Millions B      | Strong, filterable HNSW                    | GraphQL, modules, hybrid built-in   | Feature-rich OSS, hybrid     |
| <b>Qdrant</b>   | OSS + Cloud  | HNSW                       | Million B       | <b>Excellent</b> payload + filterable HNSW | Rust, fast, quantization built-in   | Filtered search, performance |
| <b>Milvus</b>   | OSS + Cloud  | HNSW, IVF-PQ, DiskANN, GPU | <b>Billions</b> | Yes                                        | Most scalable OSS, many index types | Massive self-hosted          |

| DB                                | Type          | Index                        | Scale      | Filtering | Highlights                         | Best for                 |
|-----------------------------------|---------------|------------------------------|------------|-----------|------------------------------------|--------------------------|
| <b>FAISS</b>                      | Library       | Flat, HNSW, IVF, IVF-PQ, OPQ | Billion    | DIY       | Max control, GPU, all algorithms   | Research, custom infra   |
| <b>ScaNN</b>                      | Library       | Anisotropic + tree           | Billions   | DIY       | Fastest in-memory MIPS (Google)    | Speed-critical MIPS      |
| <b>Annoy</b>                      | Library       | RP-tree forest               | Millior    | DIY       | mmap, immutable, simple            | Static read-only, recsys |
| <b>Elasticsearch / OpenSearch</b> | Search engine | HNSW (Lucene)                | Millions/B | Native    | Best hybrid (BM25 + vector in one) | Already on ES, hybrid    |

### 10.3 Managed vs self-hosted

|                       | Managed (Pinecone, Weaviate Cloud, Qdrant Cloud) | Self-hosted (Milvus, Qdrant, FAISS)     |
|-----------------------|--------------------------------------------------|-----------------------------------------|
| <b>Ops burden</b>     | Near zero                                        | You run sharding, replication, upgrades |
| <b>Cost</b>           | \$ per pod/usage (can be high at scale)          | Hardware + eng time                     |
| <b>Control</b>        | Limited tuning                                   | Full (custom indexes, GPU)              |
| <b>Data residency</b> | Their cloud (compliance concerns)                | Your infra                              |
| <b>Scale ceiling</b>  | Very high (their problem)                        | Yours to engineer                       |

**Interview takeaway:** Default recommendation — “If we're on Postgres and < ~10M vectors, pgvector. For fast time-to-production, Pinecone. For self-hosted scale with filtering, Qdrant or Milvus. For maximum control / research / GPU / billion-scale custom, FAISS. If we already run Elasticsearch and need hybrid, use its native kNN.”

## 11 Scaling, Updates & Operations

### 11.1 Sharding

At billions of vectors, one machine can't hold the index. Shard the vectors across nodes; query all shards in parallel; merge the per-shard top-k.

| Strategy                               | How                                                | Pro                                       | Con                                                  |
|----------------------------------------|----------------------------------------------------|-------------------------------------------|------------------------------------------------------|
| <b>Shard by id (random/hash)</b>       | hash(id) % num_shards                              | Even load, simple                         | <b>Every query hits every shard</b> (scatter-gather) |
| <b>Shard by cluster (IVF centroid)</b> | Route query to shards owning the nearest centroids | Query touches few shards                  | Skewed cells = hot shards; rebalancing on drift      |
| <b>Shard by tenant/metadata</b>        | One shard (or index) per tenant                    | Filtered queries hit one shard; isolation | Uneven tenant sizes; many small indexes              |



**SCATTER-GATHER (hash sharding):**

Query → [shard0 top-k][shard1 top-k]...[shardN top-k] → merge → global top-k  
 Latency = max(shard latency) + merge. Tail latency dominated by the slowest shard.

**ROUTED (cluster sharding):**

Query → nearest 2 centroids → only their 2 shards → merge.  
 Fewer shards touched → less load, but a query near a boundary may miss → lower recall.

## 11.2 Replication

Each shard is replicated (e.g., 3×) for availability and read throughput. Reads load-balance across replicas; a replica failure doesn't drop data. Writes go to a primary and propagate. Same leader-follower trade-offs as any distributed store (replication lag → a just-upserted vector may not yet be searchable on all replicas → "freshness").

## 11.3 Inserts vs deletes --- the hard part

**Inserts** are easy for HNSW (incremental graph insertion) and IVF (assign to nearest cell). The cost: HNSW insertion runs a search to find neighbors ( $\approx$  query cost), so high write rates are heavier than reads.

**Deletes** are the genuinely hard problem:

- › HNSW graphs don't support clean node removal — deleting a node would orphan its neighbors' links and degrade navigability.
- › The standard solution is **tombstones (soft delete)**: mark the vector deleted, keep it in the graph, filter it out of results at query time.
- › Tombstones accumulate → wasted memory + degraded recall (the graph routes through dead nodes) → periodic **rebuild/compaction** to physically remove them, analogous to LSM-tree compaction.

Delete v: mark v.deleted = true (tombstone; still in graph, filtered at query)  
 When tombstone ratio > ~20-30% → rebuild index from live vectors (offline or shadow)

IVF handles updates better (just drop the id from a cell's list), but accumulated churn unbalances cells → eventual retrain.

## 11.4 Filtered search at scale (recap with the scaling angle)

The pre/post-filter problem (from the RAG section) gets worse at scale. For **selective** filters (few matches), exact brute force over the matching subset beats ANN. For **non-selective** filters, filtered-HNSW or partitioning. Partitioning by the common filter key (tenant, time bucket) is often the cleanest: each partition is its own small index, queried directly — no recall loss.

## 11.5 Re-embedding when the model changes (full rebuild)

This is the operational elephant. A new embedding model = new geometry = **every vector must be recomputed and the index rebuilt**. Old and new vectors are not comparable; you cannot incrementally migrate.

**Blue/Green re-embed (zero-downtime):**

1. Stand up a NEW index (green) alongside the live one (blue).
2. Re-embed the whole corpus with the new model → populate green.
3. Validate: run the golden query set against green; require recall@10 ≥ blue.
4. Dual-write new ingests to both during the transition.
5. Cut traffic over to green; decommission blue.

Cost (100M chunks): ~28 GPU-hours embed + index build + 2× storage during overlap.

## 11.6 Freshness

How fast must a new document be searchable? Real-time chat memory needs seconds; a docs corpus can tolerate hours. Patterns: a small **hot index** (recent inserts, frequently rebuilt or HNSW with fast inserts) merged with a large **cold index** (rebuilt nightly); queries hit both and merge. Trade freshness against rebuild cost.

**Interview takeaway:** "Deletes via tombstones + periodic rebuild; inserts incremental but ~as costly as a query; model upgrades force a full blue/green re-embed — that's the biggest operational risk, budget ~28 GPU-hours per 100M chunks." Saying this signals real operational experience.

## 12 Evaluation

You cannot tune recall/latency/memory without measuring. Build a **golden set** and track retrieval metrics offline, then confirm online.

### 12.1 Building a golden set

A golden set is a list of (query, relevant\_doc\_ids) pairs — the ground truth of what *should* be retrieved.

Sources: hand-labeled by experts; mined from click logs (clicked results = relevant); or **synthetically generated** (LLM reads each chunk and writes a question it answers → (question, chunk\_id) is a positive pair). Aim for a few hundred to a few thousand queries spanning head (common) and tail (rare) cases.

### 12.2 Retrieval metrics

| Metric             | Formula / idea                                             | Captures                               |
|--------------------|------------------------------------------------------------|----------------------------------------|
| <b>Recall@k</b>    | (relevant retrieved in top-k) / (total relevant)           | Did we <i>find</i> the right docs?     |
| <b>Precision@l</b> | (relevant in top-k) / k                                    | Are top-k <i>clean</i> (not noisy)?    |
| <b>MRR</b>         | mean of 1/rank of the first relevant result                | How high is the <i>first</i> good hit? |
| <b>nDCG@k</b>      | DCG / ideal DCG, with graded relevance & position discount | Ranking quality with graded relevance  |
| <b>Hit@k</b>       | fraction of queries with $\geq 1$ relevant in top-k        | Coarse "did we get anything right?"    |

MRR example: query A first-relevant at rank 1 → 1/1  
 query B first-relevant at rank 3 → 1/3  
 query C first-relevant at rank 2 → 1/2  
 $MRR = (1 + 0.333 + 0.5) / 3 = 0.611$

nDCG@k:  $DCG = \sum_i rel_i / \log_2(i+1)$  (rel can be graded 0/1/2/3)  
 $nDCG = DCG / IDCG$  (DCG of the perfect ranking). Range [0,1].  
 Rewards putting the most-relevant doc at the top, not just somewhere in top-k.

```

1 def recall_at_k(retrieved_ids, relevant_ids, k):
2 top = set(retrieved_ids[:k])
3 return len(top & set(relevant_ids)) / len(relevant_ids)
4
5 def mrr(list_of_retrieved, list_of_relevant):
6 total = 0.0
7 for retrieved, relevant in zip(list_of_retrieved, list_of_relevant):

```

```

8 rel = set(relevant)
9 for rank, doc in enumerate(retrieved, start=1):
10 if doc in rel:
11 total += 1.0 / rank
12 break
13 return total / len(list_of_retrieved)

```

### 12.3 ANN recall vs end-to-end recall (don't conflate)

Two different "recall" numbers:

- **ANN recall@k** — does the index return the *exact* nearest vectors? (Index quality; ground truth = brute force.)
- **Retrieval recall@k** — did we retrieve the *truly relevant documents*? (System quality; ground truth = golden labels.)

You can have 0.99 ANN recall but poor retrieval recall if the *embedding model* (not the index) is bad — the index faithfully returns the wrong neighbors. Diagnose by comparing ANN-vs-brute-force (isolates index) against golden-set recall (isolates whole system).

### 12.4 Offline vs online

|               | Offline                                  | Online                                                    |
|---------------|------------------------------------------|-----------------------------------------------------------|
| <b>Signal</b> | Recall@k, nDCG, MRR vs golden set        | Click-through, dwell time, answer thumbs-up, task success |
| <b>Speed</b>  | Minutes                                  | Days (statistical significance)                           |
| <b>Use</b>    | Gate every change (model/index/chunking) | A/B test the winners; catch real-world gaps               |

**For full RAG** (with generation), add faithfulness/groundedness and answer-relevance (RAGAS, LLM-as-judge) — but retrieval recall is the upstream lever; fix it first.

**Interview takeaway:** "I'd build a golden set (synthetic + log-mined), track recall@k and nDCG offline to gate index/model/chunking changes, separate ANN recall from retrieval recall to localize regressions, and A/B test online for click-through and answer success."

## 13 Designing Search over 100M Documents (Worked Example)

**Prompt:** "Design semantic search / RAG over 100 million documents with sub-100 ms p95 latency."

Walk it as: clarify → estimate → embedding → chunking → index → sharding → hybrid+rerank → budget.

### 13.1 1. Clarify requirements

- 100M *documents*; after chunking, expect ~5 chunks/doc → \*\*~500M vectors\*\*.
- Latency target: p95 < 100 ms end-to-end. Recall target: recall@10 ≥ 0.95.
- Filters: tenant + ACL + date. Freshness: new docs searchable within minutes. Read-heavy (1000s QPS).

## 13.2 2. Back-of-envelope sizing

Vectors: 100M docs  $\times$  5 chunks = 500,000,000 chunks  
 Model: bge-base, d=768, normalized  $\rightarrow$  use inner-product metric  
 Raw float32 storage: 500M  $\times$  768  $\times$  4 B = 1.536 TB (too big for one box's RAM)

This number forces the architecture: 1.5 TB of raw vectors  $\rightarrow$  either many HNSW machines or compress with PQ. We'll do **IVF-PQ + sharding + reranking**.

## 13.3 3. Embedding model

- **bge-base-en-v1.5 (768-dim)** — strong open-source, self-hostable (no per-call API cost at 500M scale), 256–512 token sweet spot. L2-normalize  $\rightarrow$  inner-product metric.
- Fine-tune later on mined (query, clicked-chunk) pairs if golden-set recall is weak.
- Embedding cost: 500M chunks  $\div$  1000 chunks/s/GPU = 500K GPU-sec  $\approx$  139 GPU-hours (one-time, parallelizable across GPUs).

## 13.4 4. Chunking

- **Recursive character split**, 512 tokens, 64-token overlap, prefer header/paragraph boundaries (structural for the docs that have markup).
- Store metadata: {text, doc\_id, source, page, tenant\_id, acl[], created\_at}.

## 13.5 5. Index choice + params

500M vectors  $\rightarrow$  memory is the binding constraint  $\rightarrow$  **IVF-PQ** (compress) with a **rerank** stage:

IVF: nlist =  $4 \times \sqrt{500M} \approx 4 \times 22,360 \approx 90,000$  centroids  
 $\rightarrow$  ~5,500 vectors per cell. nprobe = 64 (query-time tunable).  
 PQ: d=768, m=96 sub-vectors (8 dims each), 8-bit codes (256 centroids/sub).  
 Per-vector storage: 96 bytes (+ a few bytes overhead).  
 Memory: 500M  $\times$  ~100 B  $\approx$  50 GB of PQ codes  $\leftarrow$  now fits across a few machines.  
 Rerank: keep full float vectors on NVMe SSD; for the top-200 IVF-PQ candidates,  
 re-score with exact inner product  $\rightarrow$  recovers recall@10 to ~0.96.

Compared with HNSW-flat which would need  $\sim 500M \times 3.5 \text{ KB} \approx$  **1.75 TB RAM** (many more machines) — IVF-PQ is  $\sim 35\times$  cheaper on memory, accepting a rerank step.

## 13.6 6. Sharding & replication

Shard by hash(chunk\_id) into, say, 8 shards  $\rightarrow$  ~62.5M vectors / ~6.5 GB PQ codes each.  
 Each shard: 3 replicas (availability + read throughput).  
 Query = scatter-gather: fan out to 8 shards, each returns top-50, merge  $\rightarrow$  top-200,  
 then rerank. Latency  $\approx$  max(shard) + merge + rerank.  
 For tenant isolation, optionally co-locate a tenant's chunks (route filtered queries to fewer shards). Large tenants get dedicated indexes.

## 13.7 7. Hybrid + rerank pipeline

```

Query
├─ embed (bge-base) $\rightarrow$ IVF-PQ ANN (nprobe=64) per shard $\rightarrow$ merge top-200 [~25 ms]
├─ BM25 (Elasticsearch/Lucene) $\rightarrow$ top-100 [~15 ms, parallel]
├─ RRF fuse (k=60) $\rightarrow$ top-100
├─ Cross-encoder rerank (ms-marco MiniLM) on 100 $\rightarrow$ top-5 [~40 ms]
└─ (RAG) inject top-5 into LLM prompt with citations

```

## 13.8 8. Latency & resource budget

|    |                                           |                                    | Python |
|----|-------------------------------------------|------------------------------------|--------|
| 1  | Embed query (1 forward pass, GPU/CPU)     | ~5 ms                              |        |
| 2  | IVF-PQ scatter-gather (8 shards parallel) | ~25 ms (p95; tail = slowest shard) |        |
| 3  | BM25 (parallel, overlaps)                 | ~15 ms                             |        |
| 4  | RRF fuse                                  | ~2 ms                              |        |
| 5  | SSD rerank of top-200 (exact float)       | ~10 ms                             |        |
| 6  | Cross-encoder rerank top-100              | ~40 ms                             |        |
| 7  |                                           |                                    |        |
| 8  | End-to-end p95                            | ~80 ms ✓ under 100 ms              |        |
| 9  | Recall@10 (post-rerank)                   | ~0.96 ✓                            |        |
| 10 | Memory (PQ codes, replicated 3x)          | ~150 GB total (50 GB × 3)          |        |
| 11 | One-time embed                            | ~139 GPU-hours                     |        |

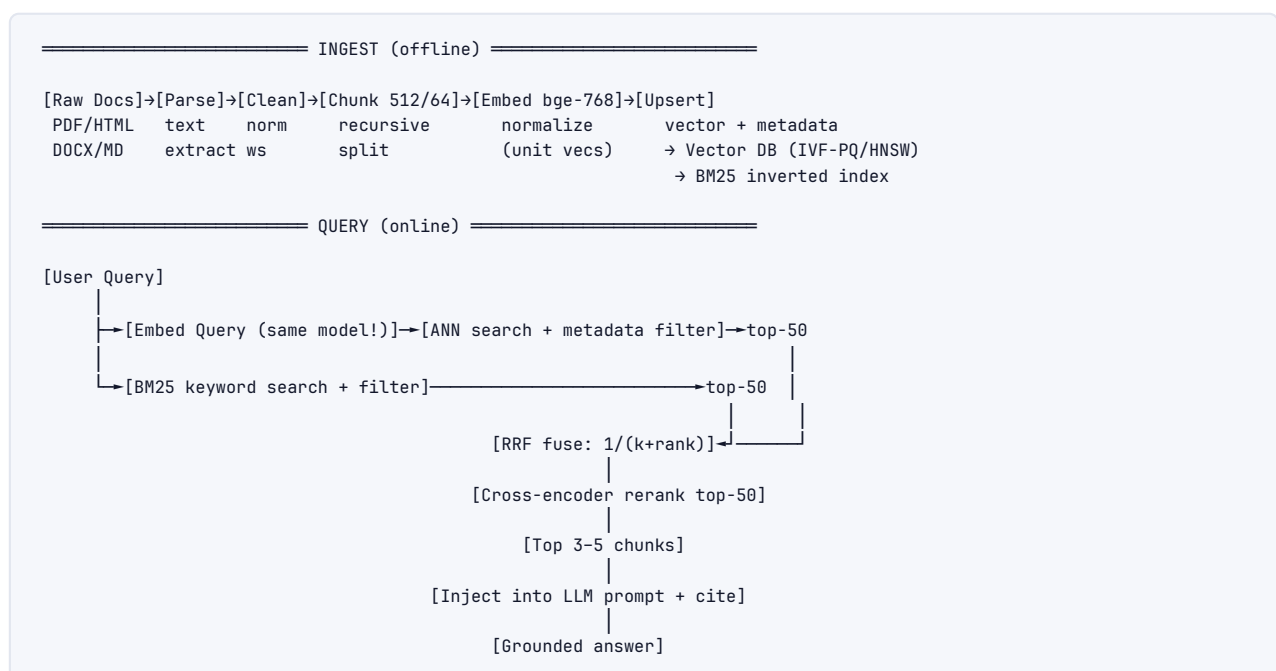
## 13.9 9. Trade-offs to state aloud

- › **IVF-PQ vs HNSW:** chose PQ for ~35× memory savings; pay with a rerank stage and a k-means training step (retrain on drift). If memory weren't constrained, HNSW (M=32) would give ~0.97 recall with no rerank but ~1.75 TB RAM.
- › **nprobe / ef\_search** are the live recall-vs-latency knobs.
- › **Filtering:** filtered-IVF or tenant partitioning; exact brute force for highly selective ACL filters.
- › **Model upgrade:** blue/green full re-embed (~139 GPU-hours), validate on golden set, cut over.
- › **Freshness:** small hot HNSW index for recent inserts merged with the big IVF-PQ cold index; nightly compaction.

**Interview takeaway:** The number 500M vectors → 1.5 TB raw is what justifies every later decision (PQ, sharding, rerank). Derive it early and let it drive the design.

## 14 Architecture / Diagrams

### 14.1 Full RAG Pipeline (ingest + query)



## 14.2 HNSW search

QUERY  $q = [0.12, -0.43, 0.87, \dots]$

Layer 2 (sparse, long links): [N1] — [N8]

Layer 1 (medium): [N1] — [N3] — [N5] — [N8]

Layer 0 (all nodes, local): [N1] — [N2] — [N3] — [N4] — [N5] — [N6] — [N7] — [N8] — [N9]

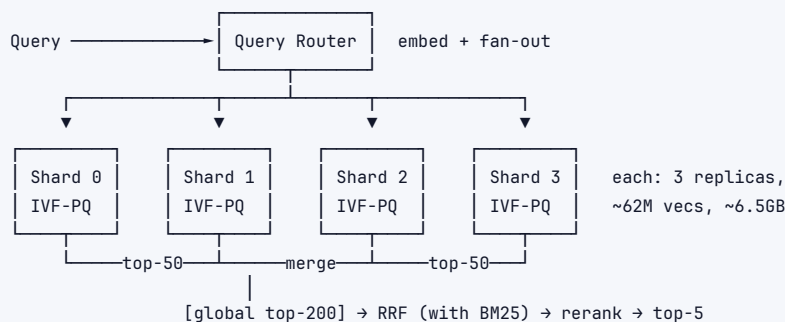
1. Enter top layer at N1, greedily hop toward  $q \rightarrow N8$
  2. Descend to L1, hop  $\rightarrow N5$
  3. Descend to L0, expand candidate set (ef\_search), refine  $\rightarrow$  top-k
- Result: approx nearest neighbors in  $\sim O(\log N)$  hops.

## 14.3 IVF-PQ retrieval

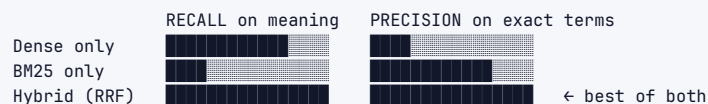
Query  $q$

- ▼ find nprobe nearest centroids (coarse quantizer)  
[C12][C57][C90][...] ← nprobe=64 cells chosen out of nlist=90,000
- ▼ scan only those cells' PQ codes (96 bytes each) via ADC lookup table  
[approx top-200]
- ▼ (optional) rerank with exact float vectors from SSD  
[top-10]

## 14.4 Sharded vector service



## 14.5 Vector vs lexical vs hybrid



# 15 Real-World Examples

## 15.1 Google Search / YouTube (ScaNN)

Embedding-based retrieval ("neural matching") finds semantically related results; ScaNN's anisotropic quantization serves billion-vector MIPS in milliseconds. YouTube recommendations are two-tower retrieval (user tower, video tower) → ANN over the video embedding index. **Lesson:** at Google scale, quantization quality (recall-per-byte) is the whole ballgame.

## 15.2 Meta Ads & FAISS

Ads retrieval: a request embeds user/context, FAISS returns thousands of candidate ads from billions, then a heavy ranker scores them. FAISS IVF-PQ on GPU drives the candidate-generation stage. **Lesson:** two-stage (cheap ANN retrieve → expensive rerank) is universal.

## 15.3 Spotify (Annoy)

Music recommendations: embed tracks/users, Annoy's mmap-able read-only forest serves the same static daily-built index across many machines cheaply. **Lesson:** for a static corpus, an immutable mmap index is operationally simple and memory-efficient.

## 15.4 Pinterest visual search (PinSage)

Embed pins (image + text), ANN retrieve visually/semantically similar pins over billions. **Lesson:** multimodal embeddings + ANN power "more like this."

## 15.5 Perplexity / enterprise RAG (Glean, Notion AI, Microsoft 365 Copilot)

Query → hybrid retrieve (dense + BM25) over chunked corpus → rerank → LLM with citations. Copilot adds **ACL-aware filtered search** — you only retrieve documents you're permitted to see (the pre/post-filter correctness issue is a real security requirement, not just performance). **Lesson:** filtering correctness = security boundary.

## 15.6 Semantic cache for LLMs

Embed each incoming LLM query; if a past query's vector is within cosine 0.97, return the cached answer — skip an expensive LLM call. A vector DB used as a cache. **Lesson:** vector search isn't only for documents.

## 15.7 Near-duplicate detection / dedup

Embed content, find pairs above a high similarity threshold to dedup a corpus or detect plagiarism / repost spam. **Lesson:** the same NN machinery serves dedup, not just search.

# 16 Real-Life Analogies

*One vivid frame: a vast, smart library where books are placed by meaning, not just by title.*

| Concept                             | Library Analogy                                                                                                    |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Embedding</b>                    | A book's location in a "meaning map" — books on similar topics sit on nearby shelves, regardless of title spelling |
| <b>Vector space</b>                 | The whole floor plan where proximity = topical similarity, not alphabetical order                                  |
| <b>Dimensionality (768)</b>         | The number of independent ways books can differ (topic, tone, era, difficulty, ...) — 768 such axes                |
| <b>Cosine similarity</b>            | Comparing the <i>direction</i> you walk to reach two books from the entrance, ignoring how far each is             |
| <b>Normalization (unit vectors)</b> | Putting every book at the same distance from the entrance, so only the <i>direction</i> (meaning) matters          |
| <b>Exact kNN</b>                    | Walking past <i>every</i> shelf to find the closest book — correct but you'd spend all day                         |
| <b>Curse of dimensionality</b>      | In a building with 768 floors, every book ends up "near a wall," so shortcuts (skip whole wings) stop working      |

| Concept                            | Library Analogy                                                                                                                   |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>ANN</b>                         | A clever guide who takes you <i>almost</i> always to the right shelf in seconds, occasionally missing by one                      |
| <b>HNSW</b>                        | Express elevators (top floors, few stops) to get you to the right wing fast, then local stairs to the exact shelf                 |
| <b>IVF (clusters)</b>              | The library is divided into themed rooms; you only enter the few rooms nearest your topic                                         |
| <b>nprobe</b>                      | How many themed rooms you're willing to walk into before settling for what you found                                              |
| <b>Product Quantization</b>        | Replacing each book with a compact index card describing it — 32× lighter to carry, slightly fuzzy                                |
| <b>Reranking (cross-encoder)</b>   | After a runner brings 50 candidate books, the expert librarian reads each against your exact question and hands you the best 5    |
| <b>Hybrid search</b>               | Using both the meaning-map <i>and</i> the old title catalogue, because some patrons ask by exact title (an ISBN) and some by vibe |
| <b>Chunking</b>                    | Cutting fat books into chapters so the runner can hand you just the relevant chapter, not the whole tome                          |
| <b>Metadata filter</b>             | "Only books from the law section that you're cleared to read" — applied while searching, not after                                |
| <b>Tombstone delete</b>            | A "withdrawn" sticker on a book still on the shelf — ignored by patrons until the next big reshelving                             |
| <b>Re-embedding (model change)</b> | The library adopts a new classification system — every book must be re-placed and the whole map redrawn                           |
| <b>Recall@k</b>                    | Of the 10 best books that exist for your question, how many the guide actually brought you                                        |

## 17 Memory Tricks / Mnemonics

### ANN index pick — "HIP-LAS"

- › **HNSW** — default (graph, high recall, high memory)
- › **IVF** — clusters (probe nearby cells)
- › **PQ** — compress (codes, low memory)
- › **LSH** — hashing (similar collide)
- › **Annoy** — trees (mmap, immutable)
- › **ScaNN** — anisotropic (Google, fast MIPS)

**The trade-off triangle — "RLM, pick a point"** Recall / Latency / Memory — you optimize two, sacrifice the third. (Reranking lets you cheat a little.)

**Distance default — "Cosine for text; normalize and it's just dot."** Unit vectors  $\Rightarrow$  cosine = dot product = (monotone with) L2. Same neighbors.

**HNSW knobs — "M builds it, ef searches it."** M and ef\_construction set graph quality at build time; ef\_search tunes recall at query time (the live knob).

**IVF knobs — "nlist cuts the cake, nprobe takes the slices."** More nlist = smaller cells; more nprobe = more cells searched = higher recall.

**RAG retrieval order — "Chunk, Embed, Retrieve, Fuse, Rerank, Generate"  $\rightarrow$  "CERFRG"** ("Can Every Robot Find Really Good [answers]")

**RRF formula — "one over k-plus-rank, summed."**  $\sum 1/(60 + \text{rank})$ .

**Bi vs cross — "Bi is fast and blind; cross is slow and sees."** Bi-encoder embeds separately (scalable retrieval); cross-encoder reads the pair together (accurate rerank).



**The hidden cost — "New model, new map."** Change the embedding model  $\Rightarrow$  re-embed everything  $\Rightarrow$  rebuild the index.

## 18 Common Interview Questions

### 18.1 Q1: Why not just use exact kNN or a k-d tree for vector search?

**Model answer:** Exact kNN is  $O(N \cdot d)$  per query — at 100M vectors  $\times$  768 dims that's ~77B operations, multiple seconds per query, unusable. k-d trees (and other space-partition trees) work in low dimensions but collapse under the curse of dimensionality: in high-d, distances concentrate (nearest and farthest neighbors become nearly equidistant) and almost all volume is near cell boundaries, so the pruning that makes trees fast almost never fires — they degrade to near-brute-force. That's why we use ANN (HNSW/IVF/PQ): accept ~95–99% recall for a 100–1000 $\times$  speedup.

**Follow-ups:** *At what N do you switch from brute force to ANN?* (~100K–1M; below that, exact is fine and simpler.) *What's the recall cost?* (Tunable via `ef_search/nprobe`; measure on a golden set.)

### 18.2 Q2: Explain HNSW. Why is it the default?

**Model answer:** HNSW is a hierarchical, navigable small-world graph — a skip-list idea applied to nearest-neighbor search. Upper layers are sparse with long-range links (express lanes); the bottom layer holds all nodes with short local links. You enter at the top, greedily hop toward the query, descend layer by layer, and refine at layer 0 keeping `ef_search` candidates. Search is  $\sim O(\log N)$ . It's the default because it gives the highest recall at a given latency, needs no training step, supports incremental inserts, and exposes a query-time recall knob (`ef_search`). Its costs are high memory (stores full vectors plus graph edges — ~3.5 KB/vector at  $d=768$ ,  $M=32$ ) and awkward deletes (tombstones + periodic rebuild).

**Follow-ups:** *What do M, ef\_construction, ef\_search do?* (Graph degree; build-time search width; query-time search width.) *Memory for 100M vectors?* (~358 GB.) *How do you delete?* (Tombstone, filter at query, rebuild when bloat exceeds ~25%.)

### 18.3 Q3: How does IVF-PQ let you index a billion vectors?

**Model answer:** Two ideas combined. **IVF** k-means-partitions the space into `nlist` Voronoi cells; at query time you only search the `nprobe` cells nearest the query, skipping ~99% of vectors. **PQ** compresses each vector: split it into `m` sub-vectors, learn a 256-entry codebook per subspace, and store each vector as `m` 1-byte codes — e.g., 768 floats (3072 B)  $\rightarrow$  96 bytes, a 32 $\times$  reduction. At query time, asymmetric distance computation precomputes a lookup table so each candidate's approximate distance is `m` table adds. For  $1B \times 768$ : raw float32 = 3.07 TB (won't fit), PQ = 96 GB (fits). IVF avoids scanning everything; PQ makes what you do scan tiny. Add a rerank of the top candidates with exact float vectors to recover recall.

**Follow-ups:** *Why asymmetric (don't quantize the query)?* (Higher accuracy — query stays full precision.) *Downside of PQ?* (Lossy  $\rightarrow$  lower recall; needs training; retrain on drift.) *How to recover recall?* (Rerank top-N with exact distances.)

### 18.4 Q4: Cosine vs dot product vs L2 --- which and why for text?

**Model answer:** Cosine is the text default: it compares direction (meaning), ignoring magnitude (which correlates with length, not meaning). But if I L2-normalize the vectors — which most text embedding models do — then dot product equals cosine (since  $\|a\|=\|b\|=1 \Rightarrow a \cdot b = \cos$ ), and L2 produces the *same ranking* ( $\|a-b\|^2 = 2 - 2\cos$ ). So in practice I normalize once and configure the index for inner product, getting exact cosine ranking with less per-query compute. L2 is the FAISS default and fine on normalized vectors; un-normalized, cosine and

L2 differ because L2 is magnitude-sensitive.

**Follow-ups:** *When would dot product (not cosine) be right?* (Recsys where magnitude encodes popularity/confidence — MIPS.) *Prove cosine=dot for unit vectors.* ( $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos\theta = \cos\theta$ .)

### 18.5 Q5: How do you choose chunk size, and what's the trade-off?

**Model answer:** Chunks are what you embed and retrieve, so size controls precision vs. context. Too small (e.g., 64 tokens): high precision but the chunk lacks enough context to answer ("Click Settings, then Security" without the rest of the steps). Too large (e.g., 2000 tokens): the relevant sentence is diluted by irrelevant text — the averaged embedding is fuzzy (low precision) and you waste the LLM's context window. The sweet spot is 256–512 tokens with 10–20% overlap so a sentence split at a boundary still appears whole somewhere. For structured docs I split on headers; for high precision I use parent-child (retrieve a small child, feed the larger parent to the LLM).

**Follow-ups:** *What's overlap for?* (Avoid cutting an answer across a boundary.) *Parent-child?* (Embed small chunks for precise retrieval, but pass the surrounding parent to the LLM for context.)

### 18.6 Q6: Walk me through filtered search. What's the gotcha?

**Model answer:** The gotcha is **post-filtering**: if you ANN-retrieve top-k and *then* drop non-matching results, a selective filter (say 1% of vectors match) can leave you with zero results even though good matches exist — a correctness bug — and over-fetching to compensate is slow. Naive **pre-filtering** breaks HNSW because the graph navigates through nodes that may be filtered out, collapsing recall. The production answer is **filtered ANN**: the predicate is fused into graph traversal (Qdrant, Weaviate, Milvus do this), or you partition by the filter key (one index per tenant), or — for highly selective filters — fall back to exact brute force over the tiny matching subset. In ACL-aware enterprise RAG this isn't just performance; it's a security boundary.

**Follow-ups:** *How does partitioning help?* (Each partition is its own small index, queried directly — no recall loss.) *When is brute force fine?* (When the filter leaves few candidates.)

### 18.7 Q7: Why hybrid search, and how do you combine the two rankings?

**Model answer:** Dense embeddings excel at paraphrase/concept matching but miss exact strings — SKUs, error codes, rare proper nouns the model never saw. BM25 nails those exact terms but misses synonyms. Neither alone is best, so I run both and fuse with **Reciprocal Rank Fusion**: score each doc by  $\sum 1/(k+\text{rank})$  across the two lists ( $k \approx 60$ ). RRF uses only ranks, so it sidesteps the problem that BM25 scores and cosine scores aren't comparable. A doc both methods rank highly wins; the method works without score normalization or tuning.

**Follow-ups:** *Why rank-based not score-based fusion?* (Scores live on incomparable scales; ranks are robust.) *What does k control?* (Higher k flattens the contribution of top ranks — smoother fusion.)

### 18.8 Q8: Explain retrieve-then-rerank. Why two stages?

**Model answer:** Retrieval uses a bi-encoder — query and documents embedded independently — which is fast and scalable (embed the corpus once offline, ANN at query time) but coarse, since the query and document never attend to each other. A cross-encoder feeds [query, doc] together through full attention and outputs a precise relevance score, but it needs one forward pass per document — impossible over 100M docs. So: stage 1 retrieves top-30–50 cheaply with the bi-encoder ANN; stage 2 reranks just those 30–50 with the cross-encoder and keeps the top 3–5. You get the cross-encoder's accuracy at the bi-encoder's scale. Reranking ~50 docs adds ~40 ms but materially improves answer quality.

**Follow-ups:** *Why not cross-encode everything?* ( $O(N)$  forward passes — billions of transformer runs.) *Commercial options?* (Cohere/Voyage/Jina rerankers.)

### 18.9 Q9: You upgrade your embedding model. What happens operationally?

**Model answer:** A new model produces a different vector space — old and new vectors aren't comparable — so you must **re-embed the entire corpus and rebuild the index**; you can't mix or incrementally migrate. I'd do blue/green: stand up a new (green) index, re-embed everything into it, validate recall@10 on the golden set against the live (blue) index, dual-write new ingests during transition, then cut over and decommission blue. Budget roughly 28 GPU-hours per 100M chunks plus  $2\times$  storage during overlap. This re-embed cost is the single biggest hidden operational risk of a vector system.

**Follow-ups:** *Can you avoid re-embedding?* (No — geometry changes. Matryoshka embeddings let you truncate dims without re-embedding, but a *new model* still requires it.) *How do you validate the new index?* (Golden-set recall must meet or beat the old.)

### 18.10 Q10: How do you measure whether retrieval is good?

**Model answer:** Build a golden set of (query, relevant\_doc\_ids) — from expert labels, click logs, or LLM-generated questions per chunk. Then track recall@k (did we find the relevant docs?), precision@k (are top-k clean?), MRR (how high is the first hit?), and nDCG@k (graded ranking quality). Crucially, separate **ANN recall** (index returns the true nearest vectors, ground-truthed by brute force) from **retrieval recall** (we return the truly relevant documents, ground-truthed by labels): a bad embedding model gives perfect ANN recall but poor retrieval recall. Gate every index/model/chunking change on offline metrics, then A/B test online for click-through and answer success.

**Follow-ups:** *Define nDCG.* ( $DCG = \sum rel_i / \log_2(i+1)$ , normalized by ideal DCG; rewards ranking the best doc first.) *Recall 0.99 but bad answers — why?* (Embedding model or chunking, not the index.)

### 18.11 Q11: Design semantic search over 100M documents with <100 ms p95.

**Model answer:** 100M docs  $\times$  ~5 chunks  $\approx$  500M vectors. At d=768 float32 that's 1.5 TB raw — too big for one box, so memory drives the design. Use bge-base (768-dim, self-hosted, normalized  $\rightarrow$  inner product), recursive 512/64-token chunks with metadata. Index: IVF-PQ (nlist $\approx$ 90K, nprobe=64, m=96 8-bit codes  $\rightarrow$  ~50 GB codes) plus a rerank of top-200 against exact float vectors on SSD. Shard by hash into 8 shards (~62M each), 3 replicas; scatter-gather and merge. Add BM25, fuse with RRF, then cross-encoder rerank to top-5. Budget: ~5 ms query embed + ~25 ms ANN + ~15 ms BM25 (parallel) + ~10 ms SSD rerank + ~40 ms cross-encoder  $\approx$  80 ms p95, recall@10  $\approx$  0.96, ~150 GB RAM (3 $\times$  replicated). If memory weren't constrained I'd use HNSW (M=32) for ~0.97 recall and skip the rerank, at ~1.75 TB RAM.

**Follow-ups:** *Why IVF-PQ over HNSW here?* (35 $\times$  memory savings; HNSW = 1.75 TB.) *How do filters/freshness work?* (Tenant partition + filtered IVF; hot HNSW index for recent inserts merged with cold IVF-PQ.)

### 18.12 Q12: How do deletes work in a vector index?

**Model answer:** Deletes are the hard operation. HNSW graphs can't cleanly remove a node without orphaning neighbors' links, so the standard approach is a **tombstone**: mark the vector deleted, leave it in the graph, and filter it out of query results. Tombstones accumulate — wasting memory and degrading recall because the graph still routes through dead nodes — so when the tombstone ratio passes ~20–30% you **rebuild** the index from live vectors (often as a shadow index, then swap), analogous to LSM-tree compaction. IVF handles it more gracefully (drop the id from a cell's posting list), though churn eventually unbalances cells and triggers a retrain.

**Follow-ups:** *Why not remove the HNSW node immediately?* (Breaks graph navigability.) *How does this resemble databases?* (Tombstones + compaction = LSM-tree; MVCC dead tuples + VAC-

UUM.)

## 19 Senior-Level Discussion Points

### 19.1 Memory is usually the binding constraint, not CPU

At scale, the question "will the index fit in RAM?" dictates the architecture. HNSW stores full vectors (~3.5 KB/vec at  $d=768$ ) → great recall, but  $100M = 358$  GB and  $1B = 3.5$  TB. PQ trades recall for a  $32\times$  memory cut. The senior move is to start from the raw-size estimate ( $N \times d \times 4$  B) and let it choose between HNSW (memory available) and IVF-PQ (+ rerank) (memory bound). Disk-based ANN (DiskANN, Milvus) pushes vectors to NVMe with a small in-RAM graph for even larger scale at higher latency.

### 19.2 Two-stage retrieval is universal

Cheap-recall then expensive-precision appears everywhere: ANN retrieve → cross-encoder rerank; IVF-PQ approx → exact float rerank; candidate generation → heavy ranker (ads/recs). Always retrieve more than you show. The cost asymmetry (cheap stage over millions, expensive stage over dozens) is the whole point.

### 19.3 Embedding quality dominates index quality

Teams obsess over `ef_search` while their embedding model is the actual bottleneck. A perfect index over bad vectors returns the wrong neighbors faithfully. Diagnose by separating ANN recall (vs brute force) from retrieval recall (vs golden labels). Often the highest-ROI move is fine-tuning the embedding model on in-domain (query, doc) pairs, not tuning the index.

### 19.4 Hybrid + rerank beats bigger models

Before reaching for a 3072-dim model or a 7B embedder, add BM25 + RRF + a cross-encoder reranker. Hybrid fixes the exact-term blind spot of dense vectors; reranking fixes ordering. This stack usually beats a larger single bi-encoder at lower cost.

### 19.5 Filtering correctness is a security boundary

In multi-tenant / ACL systems, "retrieve only what this user can see" is not a perf nicety — leaking a retrieved chunk is a data breach. Post-filtering can both miss results (correctness) and, if implemented sloppily, leak. Use filtered-ANN or partition by tenant, and treat the filter as part of the authorization path.

### 19.6 The re-embed treadmill

Every embedding-model upgrade forces a full corpus re-embed + index rebuild. This caps how often you can chase model improvements and argues for (a) self-hosting embeddings to avoid per-call costs at re-embed time, (b) Matryoshka models to get a dim knob without re-embedding, and (c) blue/green pipelines so upgrades are routine, not heroic.

### 19.7 Freshness vs rebuild cost

Real-time use cases (chat memory) need second-level freshness; doc search tolerates hours. The hot/cold split (small frequently-rebuilt index + large nightly-rebuilt index, queried together) is the standard answer. State your freshness SLA explicitly; it changes the design.

## 19.8 Quantization recall recovery

PQ's recall hit is recoverable: retrieve a larger candidate set by PQ approx distance, then re-score with exact float vectors (kept on SSD). This decouples "fit in RAM" (PQ codes) from "final ranking accuracy" (exact rerank) — a clean way to break the recall/memory tension.

## 20 Typical Mistakes Candidates Make

1. **Saying "just use cosine similarity" with no mention of normalization or the dot-product equivalence.** The signal is knowing that normalized vectors make cosine = dot, so you configure the index for inner product.
2. **Proposing exact kNN or a k-d tree at scale.** Shows no awareness of  $O(N \cdot d)$  cost or the curse of dimensionality. Name ANN immediately for  $N > \sim 1M$ .
3. **Treating filtering as a post-step.** Post-filtering causes the empty-results correctness bug and, in ACL systems, security risk. Always raise filtered-ANN or partitioning.
4. **Confusing bi-encoder and cross-encoder.** Bi = independent embeddings for scalable retrieval; cross = joint attention for accurate reranking. Mixing them up reveals shallow RAG understanding.
5. **Ignoring chunking.** Retrieval quality is dominated by chunk size/overlap/strategy. "I'll embed the documents" without chunking is a red flag.
6. **Forgetting the re-embed cost of model upgrades.** Many candidates think you can swap embedding models freely. You can't — it's a full corpus rebuild.
7. **Conflating ANN recall with retrieval recall.** A great index over a bad embedding model still returns wrong results. Separate the two when diagnosing.
8. **"HNSW always" with no memory math.** HNSW is the default, but at billions of vectors its memory (full vectors + graph) is prohibitive; that's when IVF-PQ wins. Quote the numbers.
9. **Skipping hybrid search.** Pure dense search misses exact terms (SKUs, error codes). Mention BM25 + RRF.
10. **No golden set / no evaluation plan.** "I'd tune ef\_search" without saying *against what metric on what data* is hand-waving. Recall@k on a golden set gates everything.
11. **Claiming PQ is lossless.** PQ is lossy compression; it reduces recall. The fix is reranking with exact vectors, not pretending there's no cost.
12. **Forgetting deletes are hard.** Saying "just delete the vector" ignores HNSW's tombstone-and-rebuild reality.

## 21 How This Connects to Other Topics

| Topic                         | Connection                                                                                                                                                                                          |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LLM Applications / RAG</b> | Vector search is the retrieval engine of RAG; chunking, hybrid, rerank all live here                                                                                                                |
| <b>Databases (08)</b>         | ANN index = the B-tree analog for semantic proximity; tombstones + rebuild mirror LSM compaction + VACUUM; filtered search $\approx$ index + predicate; sharding/replication are identical concepts |
| <b>Distributed Systems</b>    | Sharding (scatter-gather vs routed), replication, consistency/freshness, quorum reads for vector shards                                                                                             |
| <b>Machine Learning</b>       | Embedding models, contrastive/InfoNCE training, bi/cross-encoders, two-tower retrieval, CLIP/multimodal                                                                                             |



| Topic                                   | Connection                                                                                                                           |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Data Structures &amp; Algorithms</b> | Skip lists (HNSW intuition), k-means (IVF/PQ), graphs (NSW), priority queues (candidate sets), k-d trees and why they fail in high-d |
| <b>System Design</b>                    | Retrieval service design, latency budgets, capacity estimation, hot/cold indexes, caching                                            |
| <b>Information Retrieval</b>            | BM25/TF-IDF, inverted indexes, RRF, nDCG/MRR/precision/recall, evaluation methodology                                                |
| <b>Performance Engineering</b>          | SIMD distance kernels, memory bandwidth limits, quantization, GPU vs CPU ANN, mmap                                                   |
| <b>Security / Privacy</b>               | ACL-aware filtered retrieval as an authorization boundary; on-device embeddings for privacy                                          |

## 22 FAANG Interview Tips

1. **Lead with the size estimate.** "N vectors  $\times$  d dims  $\times$  4 bytes" is the number that drives every architectural choice. Compute it out loud first.
2. **Always name the recall/latency/memory triangle** and which corner you're optimizing, then name the knob (ef\_search / nprobe) you'd tune against a recall@k target.
3. **Default to HNSW, switch to IVF-PQ when memory-bound.** Justify with the memory math (358 GB vs 96 GB), don't just name-drop.
4. **Never say "filter the results."** Say "filtered-ANN or partition by the filter key; exact brute force for selective filters" — and flag the post-filter correctness/security bug.
5. **Propose hybrid + rerank by default.** Dense + BM25 fused with RRF, then a cross-encoder. It beats a bigger model cheaper.
6. **Use the same embedding model for ingest and query**, normalized, inner-product metric. State the cosine=dot equivalence.
7. **Bring up the re-embed cost** whenever model choice or upgrades come up — it's the senior signal.
8. **Separate ANN recall from retrieval recall** when discussing evaluation; tie everything to a golden set.
9. **For Google, mention ScaNN/anisotropic quantization; for Meta, FAISS and two-tower; for Amazon, OpenSearch kNN and cost.** Company-specific knowledge reads as systems maturity.
10. **State trade-offs explicitly.** "PQ saves  $32\times$  memory but loses recall; I recover it by reranking the top-200 with exact float vectors on SSD." Quantified trade-offs win.

## 23 Revision Cheat Sheet

### 23.1 10-Minute Summary

**The problem:** find nearest vectors by *meaning*. Exact kNN is  $O(N \cdot d)$  (seconds at 100M); k-d trees die to the curse of dimensionality. So use **ANN** — ~95–99% recall for 100–1000 $\times$  speedup.

**Embeddings:** dense learned vectors where geometric closeness = semantic similarity. Trained contrastively (InfoNCE, in-batch negatives, SimCSE). Bi-encoder = independent embeddings (scalable retrieval); cross-encoder = joint attention (accurate rerank). Dims 384/768/1536/3072 trade quality vs memory (linear). Normalize  $\rightarrow$  cosine = dot. Model upgrade = full re-embed + rebuild.

**Metrics:** cosine for text (direction = meaning); normalized  $\Rightarrow$  cosine = dot = (monotone) L2 — same neighbors. Pick inner product on normalized vectors.

**Indexes:**

- **HNSW** — hierarchical small-world graph;  $\sim O(\log N)$ ; knobs  $M$  /  $ef\_construction$  /  $ef\_search$ ; high recall, high memory (full vectors + graph). **Default.**
- **IVF** — k-means cells; probe  $nprobe$  nearest cells; needs training.
- **PQ** — split into  $m$  sub-vectors, codebooks,  $m$ -byte codes;  $32\times$  memory cut; lossy; ADC lookup.
- **IVF-PQ** — billion-scale workhorse (FAISS).
- **LSH** (hashing), **Annoy** (mmap trees, immutable), **ScaNN** (anisotropic, Google, fastest MIPS).

**Triangle:** Recall / Latency / Memory — pick a point; tune via  $ef\_search/nprobe$ ; rerank to cheat.

**RAG pipeline:** chunk (512/64 tokens, recursive)  $\rightarrow$  embed  $\rightarrow$  upsert  $\rightarrow$  retrieve top-k  $\rightarrow$  metadata filter (filtered-ANN, not post-filter)  $\rightarrow$  hybrid (dense + BM25, fuse with RRF =  $\Sigma 1/(60 + rank)$ )  $\rightarrow$  cross-encoder rerank  $\rightarrow$  top 3–5  $\rightarrow$  LLM.

**Ops:** inserts incremental ( $\sim$ query cost); deletes = tombstone + rebuild; shard (hash scatter-gather vs cluster-routed) + replicate; re-embed on model change (blue/green); freshness via hot/cold split.

**Eval:** golden set; recall@k, precision@k, MRR, nDCG; separate ANN recall (vs brute force) from retrieval recall (vs labels); offline-gate, online A/B.

## 23.2 Key Numbers to Know

- Exact kNN:  $O(N \cdot d)$ .  $100M \times 768 \approx 77B$  ops  $\approx$  seconds/query.
- HNSW memory:  $\sim 3.5$  KB/vector ( $d=768, M=32$ )  $\rightarrow 100M \approx 358$  GB,  $1B \approx 3.5$  TB.
- PQ: 768 floats (3072 B)  $\rightarrow$  96 bytes =  $32\times$  cut.  $1B \times 768$ : 3.07 TB float32  $\rightarrow$  96 GB PQ.
- IVF:  $nlist \approx \sqrt{N} - 4 \cdot \sqrt{N}$ ;  $nprobe$  = recall knob.
- Chunks: 256–512 tokens, 10–20% overlap.
- RRF  $k \approx 60$ .
- Re-embed:  $\sim 28$  GPU-hours / 100M chunks.
- Typical recall targets: 0.90 (web) to 0.99 (medical/legal).
- Dims: 384 / 768 / 1024 / 1536 / 3072; memory linear in  $d$ .

## 23.3 Most Important Concepts for Interviews

| Rank | Concept                                                        | Why                                 |
|------|----------------------------------------------------------------|-------------------------------------|
| 1    | Why exact kNN / k-d trees fail (curse of dimensionality)       | Justifies ANN's existence           |
| 2    | HNSW mechanism + $M/ef$ params + memory                        | The default index; asked constantly |
| 3    | IVF-PQ + the memory math                                       | Billion-scale; quantization         |
| 4    | Recall/latency/memory triangle + tuning knobs                  | The core trade-off                  |
| 5    | Cosine vs dot vs L2 + normalization equivalence                | Distance fundamentals               |
| 6    | Chunking size/overlap trade-off                                | Dominates RAG quality               |
| 7    | Hybrid (dense + BM25) + RRF                                    | Beats dense-only, cheap             |
| 8    | Retrieve $\rightarrow$ rerank (bi vs cross encoder)            | Two-stage architecture              |
| 9    | Filtered search pre/post-filter gotcha                         | Correctness + security              |
| 10   | Re-embed cost on model change                                  | Senior operational signal           |
| 11   | Evaluation: recall@k/nDCG, golden set, ANN vs retrieval recall | How you know it works               |

| Rank | Concept                      | Why         |
|------|------------------------------|-------------|
| 12   | Sharding + tombstone deletes | Scaling/ops |

## 23.4 Cheat-Sheet Summary Table

| Question                        | Quick Answer                                            |
|---------------------------------|---------------------------------------------------------|
| Exact kNN cost?                 | $O(N \cdot d)$ — seconds at 100M; use ANN               |
| Why not k-d tree?               | Curse of dimensionality — pruning fails > ~20 dims      |
| Default ANN index?              | HNSW                                                    |
| Billion-scale, RAM-bound?       | IVF-PQ + rerank                                         |
| Query-time recall knob?         | HNSW <code>ef_search</code> , IVF <code>nprobe</code>   |
| Text distance metric?           | Cosine; normalize → use inner product                   |
| Cosine = dot when?              | Unit-normalized vectors                                 |
| PQ memory cut?                  | ~32× (768 floats → 96 bytes)                            |
| Chunk size?                     | 256–512 tokens, 10–20% overlap                          |
| Combine dense + BM25?           | RRF: $\Sigma 1/(60 + \text{rank})$                      |
| Rerank model type?              | Cross-encoder (joint attention)                         |
| Filter correctly?               | Filtered-ANN or partition; never naive post-filter      |
| Delete from HNSW?               | Tombstone + periodic rebuild                            |
| Model upgrade?                  | Full re-embed + rebuild (blue/green)                    |
| Retrieval metric?               | Recall@k, nDCG, MRR on a golden set                     |
| ANN recall vs retrieval recall? | Index correctness vs system correctness — separate them |
| Spotify's library?              | Annoy (mmap, immutable)                                 |
| Meta's library?                 | FAISS                                                   |
| Google's library?               | ScaNN (anisotropic quantization)                        |